

UNIVERSIDAD DE BUENOS AIRES
Facultad de Ingeniería

Organización del Computador
Ignacio Agliani

Resumen Para el Exámen Final

Cátedra Benitez

1. Unidad 1 (Solamente Teoría sobre IEEE 754):

El estándar **IEEE 754** define cómo se representan los números de punto flotante en sistemas de cómputo. Define tres formatos principales:

- **Precisión simple:** 32 bits
- **Precisión doble:** 64 bits
- **Precisión extendida:** 80 bits

Todos los formatos usan **base 2** para las fracciones y **notación de exceso** para los exponentes. La idea central es representar números en la forma $1.\text{mantisa} \times 2^{\text{exponente}}$, donde el 1 antes del punto binario es **implícito** (no se almacena).

1.1. Estructura del Formato

Cada número de punto flotante se divide en tres campos:

Campo	Precisión Simple	Precisión Doble
Signo	1 bit	1 bit
Exponente	8 bits	11 bits
Fracción	23 bits	52 bits
Total	32 bits	64 bits

Tabla 1: Comparación de campos en punto flotante (IEEE 754)

1.1.1. Signo

Un único bit: **0** para positivo, **1** para negativo.

1.1.2. Exponente

Se representa en **notación de exceso**:

- Precisión simple $\rightarrow$ **Exceso 127**
- Precisión doble $\rightarrow$ **Exceso 1023**

Los exponentes **0** y **255** (simple) o **0** y **2047** (doble) están reservados para representaciones especiales y **no se usan para números normalizados**.

1.1.3. Fracción (o Mantisa)

La fracción es la **parte decimal del número normalizado**. Como se asume que todo número normalizado comienza con 1., ese bit no se almacena, ahorrando 1 bit de precisión. Así:

- **Simple:** 23 bits almacenados.
- **Doble:** 52 bits almacenados

El valor de la fracción F siempre cumple $1 \leq F < 2$.

1.2. Ancho de Paso

El ancho de paso es la *distancia entre un número flotante y el siguiente representable* en el formato. Es importante destacar que **no es uniforme**: a mayor exponente, mayor es el ancho de paso, lo que significa que los números grandes tienen menor densidad de representación que los números pequeños.

1.3. Overflow & Underflow

Al operar con números de punto flotante pueden ocurrir dos situaciones problemáticas:

1.3.1. Overflow

Ocurre cuando el **exponente resultante excede el límite superior** permitido, tanto para mantisas positivas como negativas. El resultado tiende a $+\infty$ o $-\infty$. No existe un manejo gradual del overflow: directamente se representa como infinito.

1.3.2. Underflow

Ocurre cuando el **exponente resultante cae por debajo del mínimo** permitido, entrando en el intervalo $(-N_{\min}, 0) \cup (0, N_{\min})$ siendo $N_{\min}$ el mínimo número representable. El cero verdadero no cae en este intervalo y es una excepción.

Para manejar el underflow, el estándar IEEE 754 introduce los **números desnormalizados (subnormales)**, que permiten reducir la magnitud de un número de forma gradual hasta llegar a cero, en lugar de saltar abruptamente.

1.4. Tipos de Números Especiales

El estándar IEEE 754 define cinco categorías de números, identificadas por los valores del exponente y la fracción:

Tipo	Exponente	Fracción	Descripción
Normalizado	$0 < \text{Exp} < \text{Max}$	Cualquier patrón	Números flotantes comunes
Desnormalizado	0 (todos ceros)	$\neq 0$	Números muy cercanos a cero
Cero	0 (todos ceros)	0	Representa ± 0
Infinito	111...1 (todos unos)	0	Representa $\pm\infty$
NaN	111...1 (todos unos)	$\neq 0$	"Not a Number"

Tabla 2: Clasificación de valores en punto flotante

1.4.1. Cero (± 0)

Como el número normalizado siempre asume un 1 implícito, el cero no puede representarse directamente. Por convención, se representa con **exponente y fracción en cero**. Existen dos versiones: **+0** y **-0**, diferenciadas solo por el bit de signo.

Ejemplos:

- 0 00000000 000000000000000000000000 = +0
- 1 00000000 000000000000000000000000 = -0

1.4.2. Infinito ($\pm\infty$)

Cuando todos los bits del exponente son 1 y la fracción es 0, el valor representa $\pm\infty$ según el bit de signo. Esto permite continuar operaciones después de un overflow y seguir las reglas matemáticas del infinito.

Ejemplos:

- 0 11111111 000000000000000000000000 = $+\infty$
- 1 11111111 000000000000000000000000 = $-\infty$

1.4.3. Desnormalizados (Subnormales)

Cuando el exponente es cero, pero la fracción es distinta de cero, el número es **desnormalizado**.

Permiten representar números más pequeños que el mínimo normalizado, con menos bits significativos a medida que el número se acerca a cero:

- El mayor desnormalizado positivo es $(2 - 2^{-23}) \times 2^{-126}$ en base 10.
- El menor desnormalizado positivo 2^{-149} en base 10.

Ejemplos:

- 1 00000000 00100010001001010 10010 → valor subnormal negativo.
- 0 00000000 000000000000000000000001 → menor número subnormal positivo.
- 0 00000000 111111111111111111111111 → mayor número subnormal positivo.

1.4.4. NaN (Not a Number)

Se usan para el manejo de operaciones matemáticas con resultados indeterminados o indefinidos. Se representa con **todos los bits del exponente en 1 y fracción distinta de cero**. Existen dos tipos:

- **QNaN** (*Quiet NaN*): el bit más significativo de la fracción es **1**. Significa resultado indeterminado. Es el resultado de **operaciones aritméticamente no definidas**.
- **SNaN** (*Signalling NaN*): el bit más significativo de la fracción es **0**. Significa operación inválida. Se usa para señalar la ejecución de una **operación ilegal**.

Ejemplos:

- 0 11111111 100001000000000000000000 = QNaN
- 1 11111111 00100010001001010100100 = SNaN

1.5. Operaciones Especiales

El estándar define resultados precisos para operaciones con valores especiales:

Operación	Resultado
$n \div \pm\infty$	0
$\pm\infty \times \pm\infty$	$\pm\infty$
$\pm n \div 0$	$\pm\infty$
$\infty + \infty$	∞
Cualquier op. contra NaN	NaN
$\pm 0 \div \pm 0$	NaN
$\infty - \infty$	NaN
$\pm\infty \div \pm\infty$	NaN
$\pm\infty \times 0$	NaN

Tabla 3: Resultados de operaciones con casos especiales

2. Unidad 2: Máquina Elemental

2.1. Arquitectura Von Neumann

Hasta mediados de los años cuarenta, las computadoras se programaban manipulando switches en un panel frontal. En 1945, Von Neumann introdujo dos conceptos fundamentales:

- **Programa almacenado:** tanto los datos como las instrucciones se guardan en memoria antes de ejecutarse.
- **Ruptura de secuencia:** permite que la máquina tome decisiones automáticamente mediante saltos condicionales según resultados previos.

Una computadora con arquitectura Von Neumann está compuesta por:

- **Memoria:** almacena instrucciones y datos en celdas identificadas por direcciones.
- **Unidad aritmético-lógica (UAL):** realiza operaciones aritméticas y lógicas.
- **Unidad de control (UC):** busca, decodifica y ejecuta instrucciones.
- **Dispositivos de entrada/salida:** transfieren información entre periféricos y memoria.
- **Bus de datos:** transporta información entre los distintos componentes.

La UC coordina el funcionamiento general del sistema, delegando cálculos a la UAL y transferencias a los dispositivos de E/S.

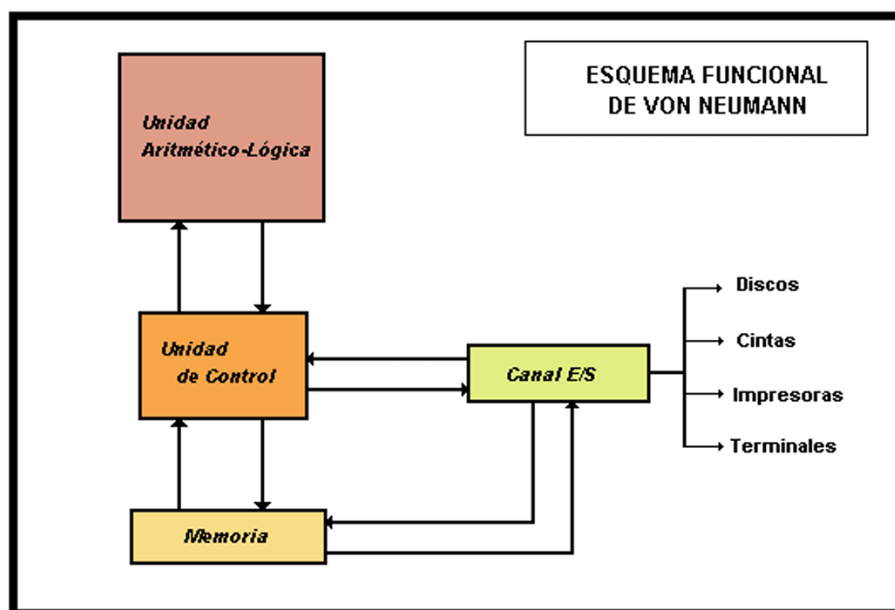


Figura 1: Esquema Funcional de Von Neumann.

2.2. Arquitectura Harvard

La **Arquitectura Harvard** se distingue por almacenar instrucciones y datos en memorias físicamente separadas. Esto permite cargar instrucciones y acceder a datos en paralelo mediante buses distintos, aumentando el ancho de banda. Aunque **dificulta la programación** al manejar espacios de direcciones separados, es muy eficiente en microcontroladores PIC y procesadores de señales digitales (DSP) para el flujo constante (streaming) de datos.

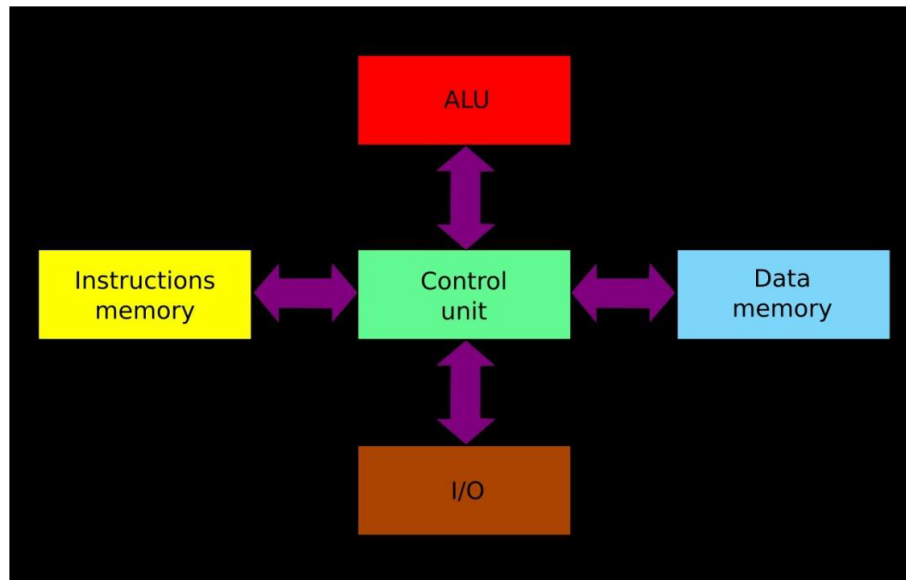


Figura 2: Esquema de la arquitectura Harvard, destacando la separación de las memorias de instrucciones y datos conectadas a la unidad de control central.

2.3. Ventajas y Desventajas

2.3.1. Arquitectura Von Neumann

Ventajas:

- **Simplicidad:** El diseño del hardware es más sencillo y económico al usar un solo bus.
- **Flexibilidad:** Permite aprovechar mejor el espacio de memoria; si un programa es pequeño, pero usa muchos datos (o viceversa), puede usar todo el espacio disponible sin desperdicio.

Desventajas:

- **Cuello de botella:** Al compartir el bus, la CPU debe esperar a que termine una tarea (ej. leer un dato) para empezar otra (ej. buscar la siguiente instrucción). Esto limita la velocidad.
- El **rendimiento** puede verse limitado cuando hay muchos accesos a memoria.

2.3.2. Arquitectura Harvard

Ventajas:

- **Mayor Velocidad:** Al tener buses separados, el procesador puede realizar varias tareas a la vez, lo que aumenta el ancho de banda y el rendimiento.
- **Predecible:** Muy útil en aplicaciones de tiempo real (como streaming de datos) porque el flujo de datos no interrumpe la búsqueda de instrucciones.

Desventajas:

- **Complejidad:** Requiere más hardware (dos buses, dos sistemas de control), lo que la hace más costosa de fabricar.
- **Programación Difícil:** Al manejar espacios de direcciones distintos para datos e instrucciones, el desarrollo de software es más complejo.

2.4. ABACUS: la Máquina Elemental

Abacus es una máquina de arquitectura Von Neumann.

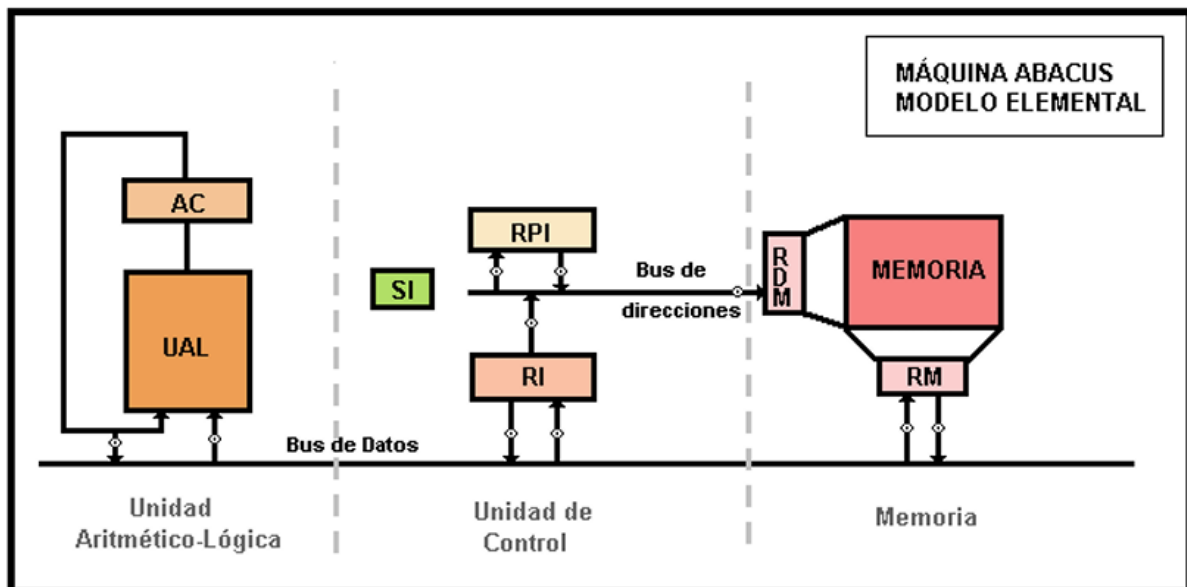


Figura 3: Esquema de la máquina elemental ABACUS.

2.4.1. Registros Principales de ABACUS

- **AC:** acumulador
- **SI:** secuenciador de instrucciones
- **RI:** registro de instrucción

- **RPI:** registro de próxima instrucción
- **RDM:** registro de direcciones de memoria
- **RM:** registro de memoria

2.4.2. Componentes de ABACUS

Abacus es una máquina de una **sola dirección**. Todas las operaciones se realizan contra el acumulador (AC), donde también queda almacenado el resultado.

Operaciones principales:

I) Unidad aritmético-lógica (UAL)

- Carga
- Almacenamiento
- Suma
- Operaciones lógicas (OR, AND, XOR)

II) Unidad de control (UC)

Controla la extracción y análisis de instrucciones.

- **RPI:** contiene la dirección de la próxima instrucción.
- **RI:** contiene el código de operación y el operando.
- **SI:** administra la apertura y cierre de compuertas.

III) Memoria

El intercambio de información se realiza mediante:

- **RDM:** tiene dirección de memoria a leer o escribir.
- **RM:** contiene dato leído o escrito.

Registro de Instrucción

El **formato de instrucción de Abacus contiene dos campos:** Código de Operación (CO) y Operando.

Al ser una máquina de una sola dirección, para sumar dos operandos en memoria es necesario ejecutar tres instrucciones:

1. Cargar el primer operando en el AC.
2. Sumar el segundo operando al contenido del AC.
3. Almacenar en memoria el resultado del AC.

2.5. Relaciones

De las relaciones entre componentes de Abacus se deducen las siguientes equivalencias:

- Tamaño RPI = Tamaño RDM = Tamaño Op = Cantidad de Celdas Direccionables
- Tamaño AC = Tamaño RI = Tamaño RM = Longitud de Instrucción = Longitud de Celda

2.6. Desarrollo de una instrucción

El desarrollo de una instrucción en Abacus se divide en **dos fases**:

- *Fase de Búsqueda:* localiza la instrucción a ejecutar. Es común a todos los tipos de instrucción y también actualiza la dirección de la siguiente instrucción.

Fase de búsqueda

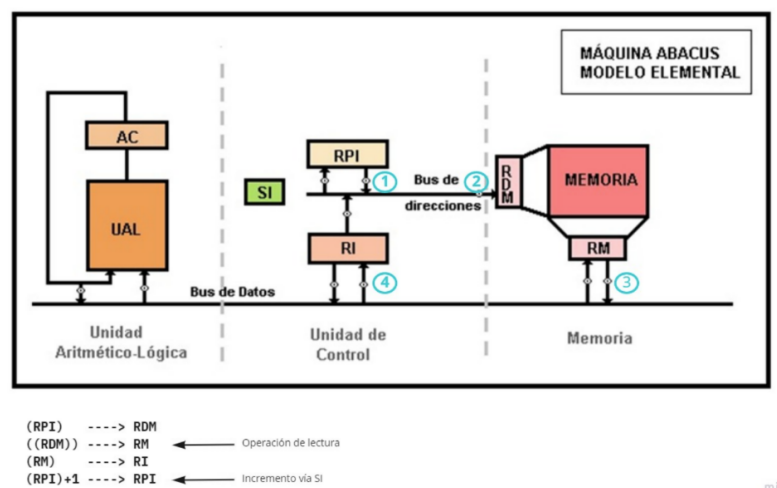


Figura 4: Fase de Búsqueda.

- *Fase de Ejecución:* ejecuta la instrucción; depende del tipo de tarea (suma, almacenamiento, salto, etc.).

2.7. Descripción de Fases

2.7.1. Búsqueda de una Instrucción

La UC transfiere el contenido del RPI al RDM y envía una orden de lectura a la memoria. El contenido de la celda queda en el RM; luego la UC transfiere el RM al RI para que los

circuitos analicen el código de operación. Finalmente se incrementa el RPI.

$$\begin{aligned} \text{RDM} &\leftarrow (\text{RPI}) \\ \text{RM} &\leftarrow ((\text{RDM})) \\ \text{RI} &\leftarrow \text{RM} \\ \text{RPI} &\leftarrow (\text{RPI}) + 1 \end{aligned}$$

Para la búsqueda de un operando, la UC transfiere el campo operando del RI al RDM y envía una orden de lectura; el operando queda disponible en el RM.

2.7.2. Ejecución de una Instrucción

En esta etapa se realiza la operación indicada por la instrucción. La UC coordina el movimiento de datos entre registros, memoria y UAL siguiendo los pulsos del reloj del sistema. La secuencia depende del tipo de instrucción.

Suma: Se suma el contenido del RM al acumulador (AC) y el resultado queda guardado en el AC.

$$\begin{aligned} \text{RDM} &\leftarrow (\text{Op}) \\ \text{RM} &\leftarrow ((\text{RDM})) \\ \text{AC} &\leftarrow \text{AC} + (\text{RM}) \end{aligned}$$

Carga: Se copia un dato desde memoria al acumulador.

$$\begin{aligned} \text{RDM} &\leftarrow (\text{Op}) \\ \text{RM} &\leftarrow ((\text{RDM})) \\ \text{AC} &\leftarrow (\text{RM}) \end{aligned}$$

Almacenamiento: El contenido del acumulador se guarda en memoria.

$$\begin{aligned} \text{RDM} &\leftarrow (\text{Op}) \\ \text{RM} &\leftarrow (\text{AC}) \\ (\text{RDM}) &\leftarrow (\text{RM}) \end{aligned}$$

Bifurcación: Permite alterar la secuencia normal del programa cargando una nueva dirección en el RPI.

$$\text{RPI} \leftarrow (\text{Op})$$

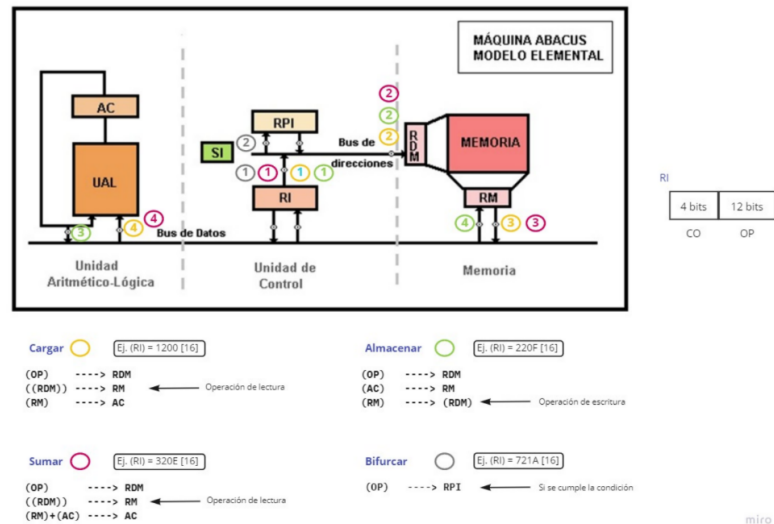


Figura 5: Fase de Ejecución.

2.8. SuperABACUS

Superabacus es una máquina de tercera generación con registros banalizados: **utilizables como registros aritméticos** o, según las condiciones de direccionamiento, **como registro base** o de índice para el cálculo de direcciones. Todos los cálculos se realizan en un único sumador que actúa a la vez de UAL y de unidad de cálculo de direcciones.

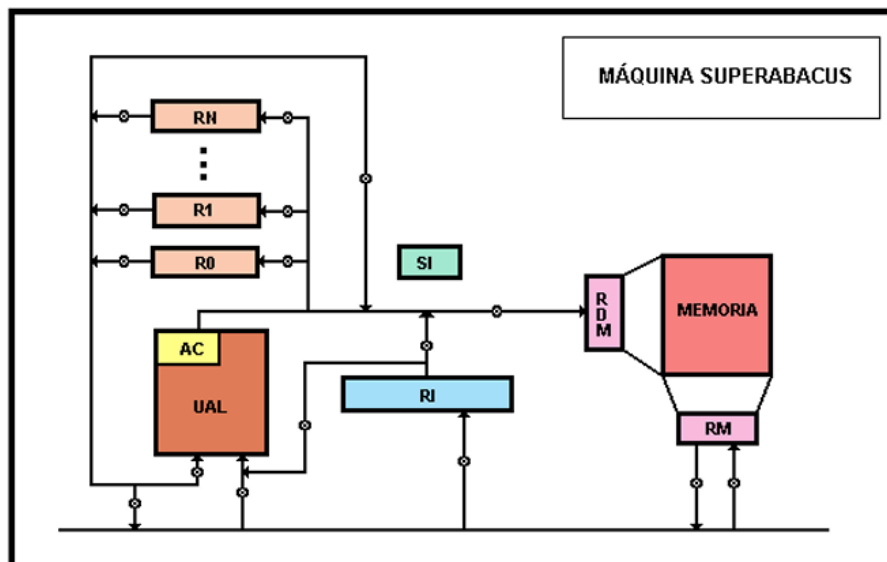


Figura 6: Esquema de la máquina SuperABACUS.

2.8.1. Características Principales de SuperABACUS

1. Posee *registros generales* que pueden contener **datos o direcciones**.
2. El registro R0 cumple la función de RPI.

3. Es una máquina de **dos direcciones**.
4. La UAL realiza cálculos aritméticos y cálculos de direcciones. La UAL cumple un doble rol porque es el único sumador de la máquina, y SuperABACUS lo aprovecha tanto para operar con datos (sumas aritméticas) como para resolver el cálculo de direcciones de memoria (incremento del PC y cómputo de desplazamientos), unificando en un solo componente lo que en otras arquitecturas requeriría hardware separado.
5. El ciclo de memoria requiere cuatro impulsos de reloj.

Gracias a esto, SuperABACUS puede ejecutar operaciones más complejas utilizando menos instrucciones.

2.8.2. Registro de Instrucción

El RI de SuperABACUS alberga instrucciones de dos operandos: Código de Operación, 1.º Operando (R_I) y 2.º Operando ($R_{II} - D$). La dirección del segundo operando se obtiene sumando el Offset al contenido del registro indicado: $A(celda) = (R_{II}) + D$.

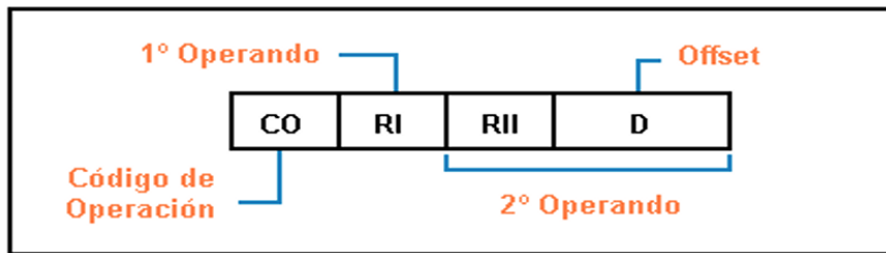


Figura 7: Formato de Instrucción de la máquina SuperABACUS.

2.8.3. Fase de Búsqueda

Al igual que en ABACUS, la fase de búsqueda es común para todas las instrucciones. La diferencia es que el incremento de R_0 (que cumple el rol de R_{PI}) se resuelve usando la UAL en vez del SI:

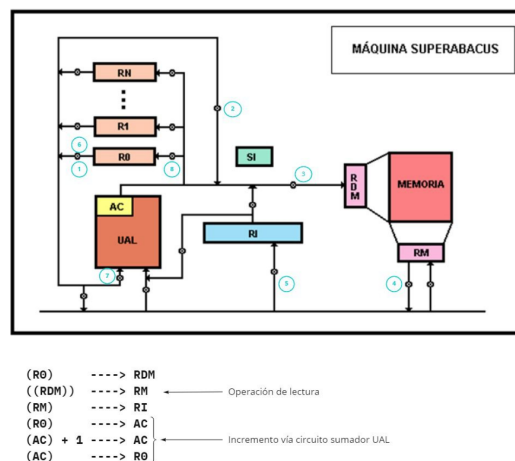


Figura 8: Procedimiento de Fase de Búsqueda de la máquina SuperABACUS.

2.8.4. Instrucciones y Modos de Direccionamiento

Las operaciones pueden ser entre registros, entre un registro y un dato inmediato, o entre un registro y un operando en memoria. Se analiza cada caso para la instrucción **SUMAR**:

1. Sumar Registro (**Modo De Direccionamiento Registro Directo**) — **SUMAR** R_I , R_{II} (ej.: **SUMAR** 3, 4):
Se suman los contenidos de ambos registros; el resultado se almacena en el primero.

$$\begin{aligned}AC &\leftarrow (R3) \\AC &\leftarrow (R4) + (AC) \\R3 &\leftarrow (AC)\end{aligned}$$

2. Sumar Inmediato (**Modo De Direccionamiento Inmediato**) — **SUMAR** R_I , #INM (ej.: **SUMAR** 2, 100):
Se suma al registro del 1.º operando el dato inmediato de la instrucción; el resultado queda en ese registro.

$$\begin{aligned}AC &\leftarrow (R2) + 100 \\R2 &\leftarrow (AC)\end{aligned}$$

3. Sumar Palabra en Memoria (**Modo De Direccionamiento Registro Indirecto**) — **SUMAR** R_I , (R_{II}) (ej.: **SUMAR** 5, (3)):
Se suma al registro del 1.º operando el contenido de la celda cuya dirección está en el 2.º registro. Como el AC tiene entrada desde registros y desde memoria, la suma puede realizarse simultáneamente.

$$\begin{aligned}RDM &\leftarrow (R3) \\RM &\leftarrow ((RDM)) \\AC &\leftarrow (R5) + (RM) \\R5 &\leftarrow (AC)\end{aligned}$$

4. Sumar Palabra en Memoria (**Modo De Direccionamiento Base + Desplazamiento**) — **SUMAR** R_I , DI(R_{II}) (ej.: **SUMAR** 5, 20(3)):
Se suma al registro del 1.º operando el contenido de la celda cuya dirección es el contenido del 2.º registro más el Offset. El resultado se almacena en el 1.º registro.

$$\begin{aligned}AC &\leftarrow (R3) + 20 \\RDM &\leftarrow (AC) \\RM &\leftarrow ((RDM)) \\AC &\leftarrow (R5) + (RM) \\R5 &\leftarrow (AC)\end{aligned}$$

Cada variante de **SUMAR** tiene un código de operación diferente, lo que permite a SuperABACUS identificar el modo de direccionamiento a utilizar (Mnemónico $\neq$ Código Operación).

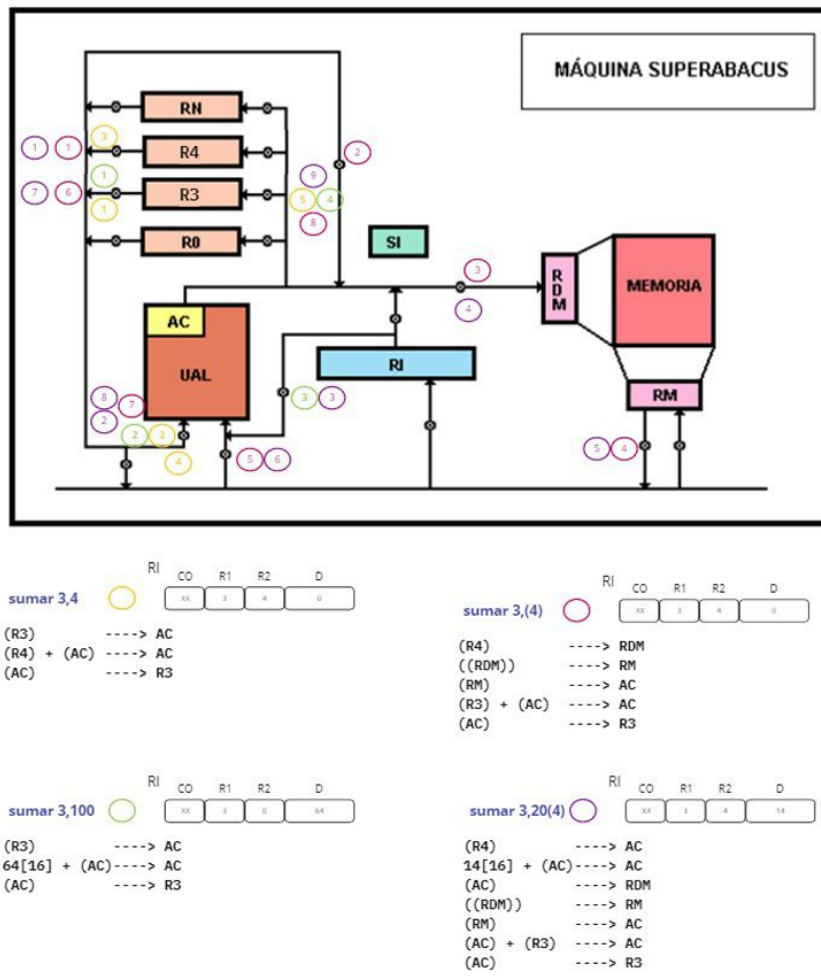


Figura 9: Comportamiento de las Compuertas de la Máquina SuperABACUS ante Distintos Tipos de Operaciones.

3. Unidad 3: Arquitectura del Conjunto de Instrucciones

3.1. Arquitectura y Organización de un Computador

La **arquitectura de computadoras** se refiere a *las características de un sistema que son visibles para el programador*, es decir, aquellos atributos que tienen un impacto directo en la ejecución lógica de un programa. Un **ejemplo** claro de esto es si el sistema posee o no una instrucción específica para realizar multiplicaciones. Dentro de este concepto se encuentra la **ISA** (*Instruction Set Architecture*), **que abarca:** el repertorio de instrucciones, los registros, tipos de datos, modos de direccionamiento y la gestión de memoria (incluyendo el tamaño de palabra y el formato endianness).

Por otro lado, la **organización de computadoras** (o microarquitectura) es la *implementación física de dicha arquitectura*. Define cómo se interconectan las unidades operativas a través de señales de control, interfaces y tecnología de memoria. Mientras que la arquitectura define "qué" hace el equipo (ej. una multiplicación), la **organización define 'cómo' se ejecuta internamente** (ej. mediante sumas sucesivas o un circuito especializado). Una misma arquitectura puede tener distintas implementaciones u organizaciones para equilibrar costos, velocidad y consumo de energía, como ocurre con la arquitectura Intel x86 en procesadores Atom para móviles versus procesadores Xeon para servidores.

3.2. Implementación y Jerarquía

La microarquitectura puede ser **cableada** (implementada directamente en hardware mediante latches y decodificadores, común en procesadores RISC como ARM) **o microprogramada** (donde las instrucciones se ejecutan mediante un microcódigo interno, como en sistemas clásicos tipo IBM System/360 o instrucciones complejas de x86). Cuando varios modelos de computadoras comparten una misma arquitectura base, pero tienen distintas organizaciones y precios, se dice que pertenecen a una misma **familia de computadoras**.

Para entender el funcionamiento integral, se utiliza un **Modelo de Capas** que divide el sistema en siete niveles: desde el Nivel 0 (dispositivos electrónicos básicos) hasta el Nivel 6 (lenguajes orientados a problemas), pasando por niveles críticos como la lógica digital, la microarquitectura (Nivel 2) y el nivel de la ISA (Nivel 3), que sirve de puente entre el hardware y el software.

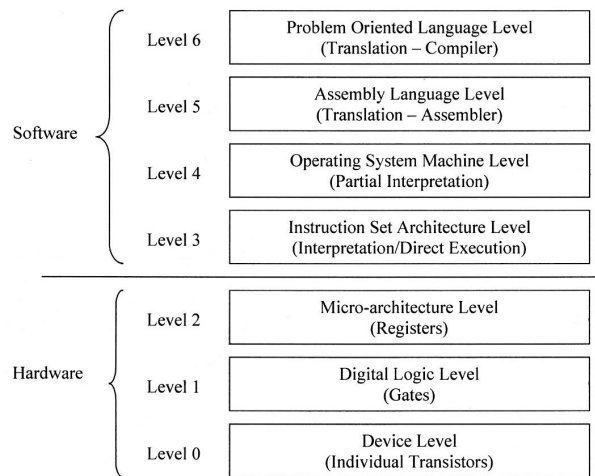


Figura 10: Representación Visual del Modelo de Capas.

3.3. Clasificación según Poder de Cálculo

3.3.1. Supercomputadoras

Son máquinas extremadamente rápidas que manejan volúmenes de datos enormes y poseen **miles de CPU**.

- Usos: Aplicaciones científicas, simulaciones y campo militar.

3.3.2. Macrocomputadoras o Mainframes

Se caracterizan por ser muy rápidas, manejar volúmenes de datos muy grandes y tener **cientos de CPU**.

- Atributo clave: Poseen una muy **alta disponibilidad**.
- **Usos:** Sistemas de gestión bancarios, telecomunicaciones e instituciones gubernamentales.

3.3.3. Minicomputadoras o Servidores Middle Range

Son equipos rápidos para volúmenes de datos grandes que poseen **decenas de CPU**.

- **Usos:** Empresas medianas y grandes, donde suelen haber varios equipos en una misma firma.

3.3.4. Microcomputadoras / PC

Diseñadas para uso individual o redes pequeñas, con **uno o varios CPU**. Manejan volúmenes de datos no muy grandes.

- **Usos:** Hogar, educación, negocios y consolas de videojuego.

3.3.5. Computadoras Portátiles

Equipos para uso individual portátil que manejan datos no muy grandes. Poseen uno o más CPU y tienen un uso hogareño principalmente.

3.3.6. Computadoras de Mano

Uso portátil acotado para acopio de datos en vía pública o información personal con bajo volumen de manejo de datos. **Incluye Smartphones, Tablets, etc.**

3.4. Introducción a la ISA

La **ISA** (*Instruction Set Architecture*), también conocida como Arquitectura de Programación, constituye la interfaz entre el hardware y el software. Esta define los elementos fundamentales que un programador necesita conocer, como:

- El repertorio de instrucciones.
- Los registros.
- Los tipos de datos.
- Los modos de direccionamiento.
- La gestión de la memoria (incluyendo el tamaño de palabra y el formato Endianness).

3.4.1. Repertorio de Instrucciones

Una **instrucción de máquina se compone** básicamente de un *código de operación* (Opcode) y de cero a n *operandos*. Estas instrucciones se clasifican en varias **categorías** según su función:

- **Aritméticas y lógicas:** Operaciones como suma, resta, multiplicación o funciones lógicas como AND y OR.
- **Movimiento de datos:** Instrucciones para cargar (**LOAD**), almacenar (**STORE**) o mover (**MOVE**) información.
- **Entrada/Salida:** Instrucciones para la comunicación con periféricos.
- **Control de flujo:** Permiten alterar la secuencia de ejecución mediante saltos (**JUMP**), llamadas a funciones (**CALL**) o comparaciones.

Los **tipos de operandos** definen dónde se encuentra el dato que la CPU va a procesar:

- **Registro:** El dato ya está dentro de los registros internos del procesador (como EAX o EBX en Intel). Es la forma más rápida de acceder a la información.
- **Memoria:** El operando es una **dirección que indica dónde está el dato** en la memoria RAM (por ejemplo, buscar el valor etiquetado como DATO).
- **Inmediato:** El **valor numérico está escrito directamente** dentro de la instrucción. No hay que buscarlo en ningún lado porque es una constante (por ejemplo, el número 5).

Estos operandos se combinan según el diseño de la ISA para ejecutar las tareas. Por ejemplo, en una instrucción de suma, podrías sumar el contenido de dos registros o un registro con un valor inmediato.

3.5. Clasificación y Formatos

La arquitectura puede clasificarse según **dónde se ubican sus operandos** (en una pila o Stack, en un Acumulador, o mediante combinaciones de Registro y Memoria) **y según el número de direcciones** que maneja la instrucción: (desde 0 hasta 3 direcciones).

La **pila** tiene 0 direcciones. Un ejemplo de instrucción es: ADD (suma el tope con el anterior).

El **acumulador** tiene 1 dirección. Un ejemplo de instrucción es: ADD A (siendo A un dato en memoria).

De 2 direcciones puede ser una instrucción **registro-memoria, registro-registro, memoria-memoria**. En este orden, algunas instrucciones son:

- ADD EAX,dword [NUMERO] (instrucción Assembly x86).
- ADD EAX,EBX (instrucción Assembly x86)
- ADD word[NUMERO1], word[NUMERO2] (instrucción no valida en Assembly x86).

O también de 3 direcciones: ADD R1,R2,R3 (instrucción Assembly ARM. Guarda en R1 la suma de R2 con R3).

En cuanto al **formato de las instrucciones** (Encoding) es el despliegue de los bits que componen la instrucción. Están **compuestos por un código de operación, operandos, modos de direccionamiento y/o flags**. Existen tres variantes principales de Encoding:

1. **Fijo:** Utilizado en arquitecturas como ARM o MIPS, donde todas las instrucciones tienen la misma longitud. ARM codifica toda instrucción en exactamente 32 bits, sin excepción. Esto es posible porque ARM sigue el esquema load/store: solo dos instrucciones (LDR y STR) tocan memoria; el resto opera exclusivamente entre registros. Al no tener que representar la enorme variedad de combinaciones registro-memoria que sí tiene x86, el **diseño puede reservar de antemano campos fijos**.
2. **Variable:** Típico de Intel x86, donde el tamaño cambia según la complejidad de la instrucción. Una **instrucción x86 se arma concatenando piezas que pueden estar presentes o no**: prefijos, opcode, desplazamiento o inmediato opcionales.

3. **Híbrido:** Un modelo intermedio utilizado en los IBM Mainframe. Todas las instrucciones de un mismo tipo miden lo mismo, pero hay ciertos tipos de instrucciones con distinta longitud.

3.6. Gestión de Datos y Memoria

El sistema debe ser capaz de manejar diversos tipos de datos, incluyendo formatos numéricos (punto fijo y flotante), caracteres (ASCII, Unicode) y direcciones. Para acceder a estos datos, **se emplean distintos modos de direccionamiento**, como el inmediato, directo, indirecto o por desplazamiento (relativo al PC o indexado).

3.7. Modos direccionamiento

- **Inmediato:** el valor numérico se escribe directamente en la instrucción. Instrucción ejemplificadora: `MOV AX,5`.
- **Memoria Directo:** Se indica la dirección exacta de memoria (o el nombre de una variable). Instrucción ejemplificadora: `MOV RCX,qword[resultado]`.
- **Memoria Indirecto:** la instrucción apunta a una posición de memoria que contiene la dirección efectiva del dato (puntero).
- **Registro:** La operación se realiza exclusivamente entre los registros internos de la CPU. Instrucción ejemplificadora: `MOV RCX,RBX`.
- **Registro Indirecto:** El registro contiene la dirección de memoria donde está el dato. Instrucción ejemplificadora: `MOV RCX,[RAX]`.
- **Desplazamiento Relativo (Program Counter):** la dirección del operando se calcula sumando un valor constante (desplazamiento) a la dirección de la siguiente instrucción que se va a ejecutar. Instrucción ejemplificadora: cualquier salto a alguna parte del programa. Podría ser: `JMP finPrograma`.
- **Desplazamiento Relativo (Registro Base):** la dirección efectiva se calcula sumando un desplazamiento constante a un registro base. Instrucción ejemplificadora: `MOV AX,[BX+4]`.
- **Desplazamiento Relativo (Indexado):** se usa una dirección base y un registro índice para acceder a elementos de arreglos o vectores.

```
1  MOV SI, 2
2  MOV AL, [BX + SI]
```

Listing 1: Desplazamiento Relativo (Indexado)

- **Stack:** es un modo de **direccionamiento implícito**. Esto significa que la instrucción no contiene ninguna dirección de memoria ni ningún nombre de registro en sus campos de operando para localizar el dato. La **CPU asume por defecto que el objetivo está en el tope**. Instrucción ejemplificadora: `POP RDX`.

Finalmente, un aspecto crítico de la memoria es el **orden de los bytes** (*Endianness*). Tomando el número 12345678_[16]:

- **Big Endian:** El byte más significativo se almacena en la dirección más baja (ej. IBM Mainframe, SPARC).

Address	Value
184	12
185	34
186	56
187	78

Figura 11: Big Endian.

- **Little Endian:** El byte menos significativo se almacena en la dirección más baja (ej. Intel x86, VAX).

Address	Value
184	78
185	56
186	34
187	12

Figura 12: Little Endian.

4. Unidad 4: Lenguaje Ensamblador

4.1. Definición

El **lenguaje de máquina** es la representación binaria de un programa, compuesta por secuencias de instrucciones que el computador interpreta directamente. Por su parte, el **lenguaje ensamblador** funciona como una representación simbólica de ese código máquina, adaptada a un procesador específico.

Address Contents			
101	0010	0010	101 2201
102	0001	0010	102 1202
103	0001	0010	103 1203
104	0011	0010	104 3204
201	0000	0000	201 0002
202	0000	0000	202 0003
203	0000	0000	203 0004
204	0000	0000	204 0000

(a) Binary program

Address Contents	
101	2201
102	1202
103	1203
104	3204
201	0002
202	0003
203	0004
204	0000

(b) Hexadecimal program

Address Instruction		
101	LDA	201
102	ADD	202
103	ADD	203
104	STA	204
201	DAT	2
202	DAT	3
203	DAT	4
204	DAT	0

(c) Symbolic program

Label	Operation	Operand
FORMUL	LDA	I
	ADD	J
	ADD	K
	STA	N
I	DATA	2
J	DATA	3
K	DATA	4
N	DATA	0

(d) Assembly program

Figure 13.13 Computation of the Formula $N = I + J + K$

Figura 13: Diferentes Representaciones del Cálculo $N = I + J + K$

A pesar del avance de los lenguajes de alto nivel, el ensamblador sigue siendo fundamental hoy en día para:

- **Optimización:** Mejorar el código en cuanto a velocidad y tamaño de memoria.
- **Control del Hardware:** Desarrollar drivers, sistemas embebidos y acceder a instrucciones que no están disponibles en lenguajes superiores.
- **Infraestructura:** Crear compiladores, realizar debugging y verificaciones críticas del sistema.

4.2. Elementos y Estructura

Un programa en ensamblador se compone de elementos básicos como **etiquetas**, **mne-mónicos**, **operandos** y **comentarios**. Dentro del código, podemos encontrar diferentes *tipos de sentencias*:

1. **Instrucciones:** es una **operación específica de programa**, formada por un mнемico y operandos. Va a estar contenida en el código objeto del programa. Ejemplo de Intel x86: `mov rax,rbx`.
2. **Directivas (pseudoinstrucciones):** son **órdenes para el ensamblador** acerca de que es o que tiene que hacer con un bloque específico de código. Estas no quedan en el código objeto. Por ejemplo, definir una sección. Ejemplo de x86: `section .data`.
3. **Macroinstrucciones:** es una **secuencia de instrucciones predefinida**, usada para evitar código repetido. El ensamblador cuando detecta una llamada a una macro, remplaza la llamada por el contenido de la macro. Ejemplo:

```
1 %macro SUMAR 2
2 add %1,%2
3 %endmacro
```

Listing 2: Ejemplo de Macro ASM-x86

4.3. Traducción, Interpretación y Ensambladores

Existen dos formas principales de ejecutar programas de usuario:

- **Traducción:** Un programa (compilador o ensamblador) convierte el código fuente en un lenguaje destino (ej. C, Rust o ensambladores como x86 y ARM).
- **Interpretación:** Un programa ejecuta directamente el código fuente (ej. Python, JavaScript) o utiliza un código intermedio como el Bytecode de Java.

Específicamente, un **ensamblador** es el programa que traduce el lenguaje ensamblador en código objeto. Existen ensambladores de *una pasada* y de *dos pasadas*. El proceso de los ensambladores de dos pasadas es el más detallado:

1. **Primera Pasada:** El ensamblador examina cada sentencia para construir una *tabla de símbolos*. Utiliza un contador de ubicación (LC) para determinar la longitud de las instrucciones y registrar las direcciones de las etiquetas y áreas de almacenamiento.
2. **Segunda Pasada (Traducción):** Se genera el código binario final. El ensamblador traduce los mnemónicos en opcodes, asigna registros o direcciones de memoria a los operandos, convierte valores inmediatos a binario y resuelve las referencias de las etiquetas usando la tabla de símbolos creada anteriormente.

4.4. Código Objeto

Una vez escrito y ensamblado un programa, el resultado del proceso es el código objeto. Este se define como la **representación en lenguaje de máquina del código fuente** programado. Es creado por un compilador o ensamblador y luego es transformado en código ejecutable por el linkeditor.

El código objeto no es, sin embargo, el producto final listo para ejecutarse. Para llegar a eso, intervienen dos herramientas adicionales: el linker y el loader.

El *linker* (o linkeditor) es un programa utilitario que combina uno o más archivos con código objeto en un único archivo que contiene código ejecutable o cargable. El *loader*, por su parte, es una **rutina** de programa que copia un ejecutable a memoria principal para ser ejecutado.

4.4.1. Dos problemas a resolver

En el proceso que va del código objeto al ejecutable final, existen dos problemas fundamentales a resolver:

1. **Direcciones externas:** existen direcciones en el código objeto que no pueden resolverse en tiempo de ensamblado, porque hacen *referencia a símbolos definidos en otros módulos*.
2. **Reubicabilidad:** no se sabe de antemano en qué lugar de la memoria se cargará el programa. Esto es necesario porque no se conoce qué otro programa habrá en memoria al mismo tiempo, y además, en entornos de multiprogramación puede producirse un swap a disco, lo que hace imprescindible poder reubicar el código.

4.4.2. Estructura Interna del Código Objeto

Un módulo de código objeto tiene una estructura interna definida, compuesta por las siguientes secciones:

- **Identificación:** nombre del módulo y longitudes de sus partes.
- **Tabla de punto de entrada:** lista de símbolos que pueden ser referenciados desde otros módulos.
- **Tabla de referencias externas:** lista de símbolos usados en el módulo pero definidos fuera de él, junto con sus referencias en el código.
- **Código ensamblado y constantes:** el contenido propiamente dicho del módulo.
- **Diccionario de reubicabilidad:** lista de direcciones que deben ser reubicadas al momento de cargar el módulo.
- **Fin de módulo.**

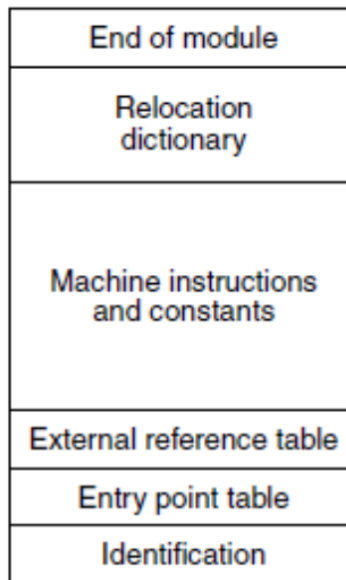


Figura 14: Estructura del Código Objeto

4.4.3. Formatos Estandarizados de Código Objeto

Existen formatos estándar para representar el código objeto, entre los más relevantes se encuentran:

- **OMF** (Object Module Format)
- **COFF** (Common Object File Format)
- **ELF** (Executable and Linkable Format)

4.5. Linking

El proceso de *linking* puede realizarse de distintas maneras, según el momento en que se resuelven las referencias entre módulos.

4.5.1. Linking Estático (Linkage Editor)

En el *linking estático*, cada módulo objeto compilado o ensamblado es creado con **referencias relativas al inicio de su propio módulo**. El *linkeditor* **combina todos los módulos objeto en un único** load module reubicable, con todas las referencias expresadas de forma relativa a ese load module.

El **proceso de generación del load module** sigue estos pasos:

1. Se construye una tabla con todos los módulos objeto y sus longitudes.
2. Se asigna una dirección base a cada módulo en función de esa tabla.

3. Se recorren todas las instrucciones que referencian a memoria y se les suma una *constante de reubicación* igual a la dirección de inicio de su módulo objeto.
4. Se recorren todas las instrucciones que referencian a otros procedimientos y se inserta su dirección correspondiente.

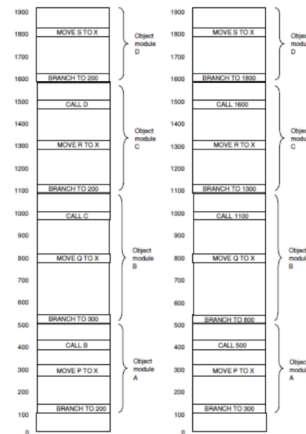


Figure 7-14. (a) The object modules of Fig. 7-13 after being positioned in the binary image but before being relocated and linked. (b) The same object modules after linking and after relocation has been performed.

Figura 15: Linking Estático

4.5.2. Linking Dinámico

El *linking dinámico* difiere la linkedición de algún módulo hasta después de la creación del load module. Existen dos variantes:

Load Time Dynamic Linking: se levanta a memoria el load module y, cuando aparece cualquier referencia a un módulo externo, el loader busca ese módulo, lo carga y cambia la referencia a una dirección relativa desde el inicio del load module. Sus ventajas son:

- Facilita la actualización de versiones del módulo externo sin necesidad de recompilar.
- El sistema operativo puede cargar y compartir una única versión del módulo externo.
- Facilita la creación de módulos de linkeo dinámico (por ejemplo, las bibliotecas .so en Unix).

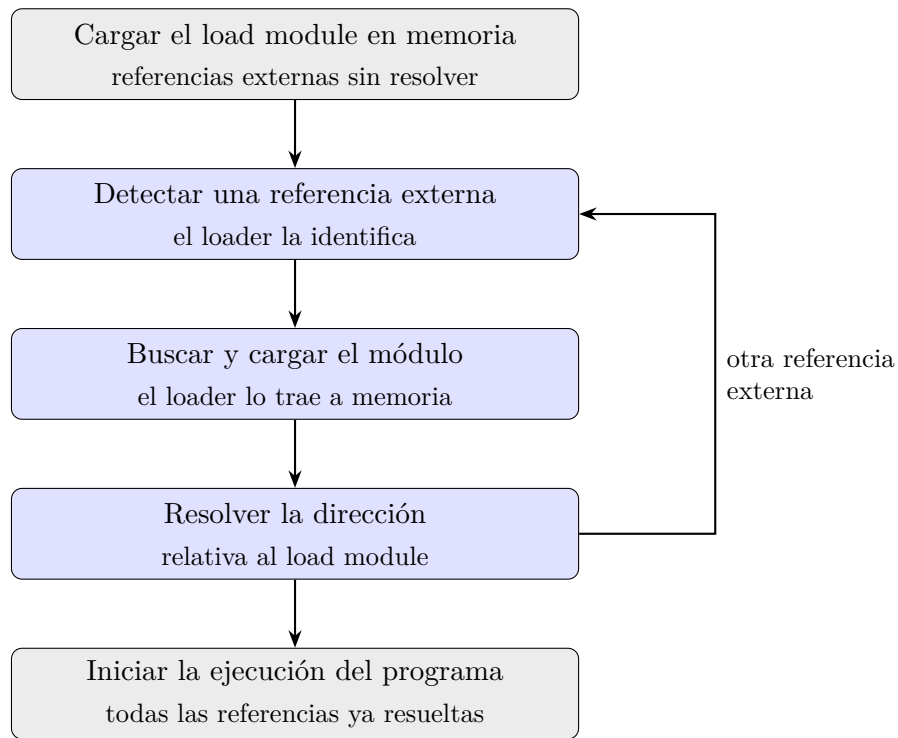


Figura 16: Load Time Dynamic Linking

Run Time Dynamic Linking: el linkeo se pospone hasta el tiempo de ejecución. Las referencias a módulos externos se mantienen en el programa cargado y, **cuando efectivamente se invoca al módulo externo, el sistema operativo lo busca**, lo carga y lo linkea al módulo llamador. Su principal ventaja es que no se ocupa memoria hasta que realmente se la necesita (por ejemplo, las bibliotecas DLL de Windows).

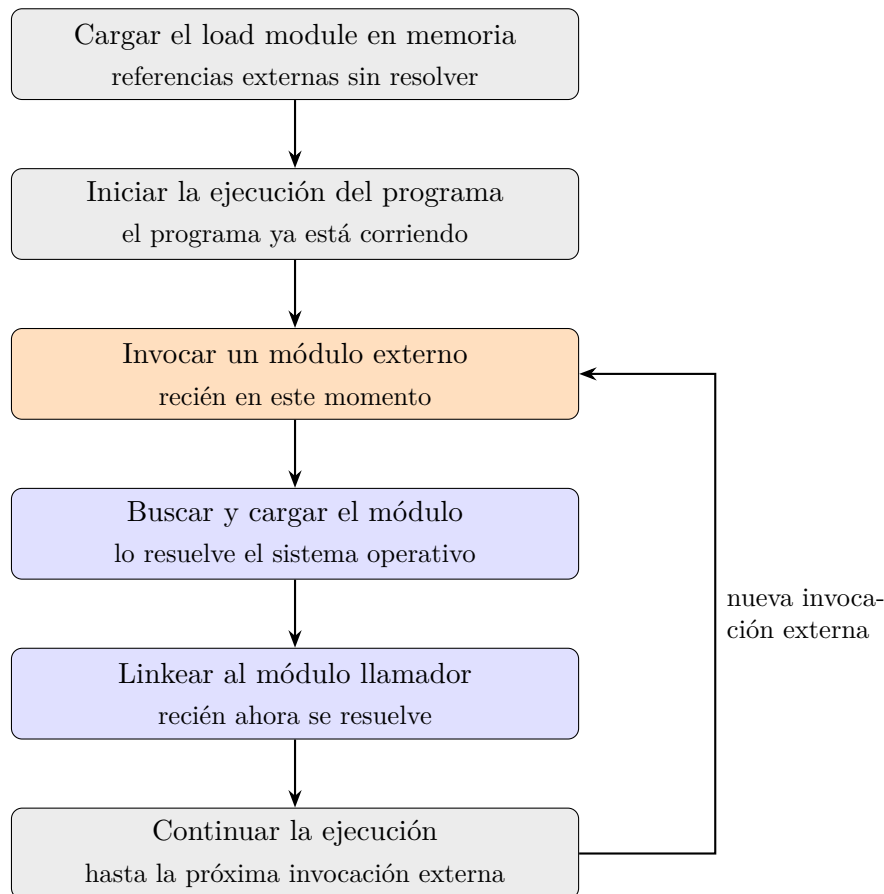


Figura 17: Run Time Dynamic Linking

4.6. Loading

El *loading* es el proceso de copiar el ejecutable a memoria principal. Existen distintas modalidades:

- **Loading absoluto:** el compilador o ensamblador genera direcciones absolutas. El módulo solo puede cargarse en un único espacio de memoria fijo.
- **Loading reubicable:** el compilador o ensamblador genera direcciones relativas al contador de locación (LC=0). El loader debe sumar un valor X a cada referencia a memoria al cargar el módulo. Para saber qué referencias modificar, el load module incluye un diccionario de reubicación.
- **Loading por registro base:** utilizado en arquitecturas con registros base para el direccionamiento. Al cargar el programa, se asigna un valor al registro base correspondiente a la ubicación en que fue cargado.
- **Loading dinámico en tiempo de ejecución:** el cálculo de las direcciones absolutas se difiere hasta el momento real de ejecución. El load module se carga a memoria con direcciones relativas, y la dirección absoluta se calcula recién cuando la instrucción va a ejecutarse efectivamente, con soporte de hardware especial.

4.7. Lenguaje Ensamblador Intel x86

4.7.1. Registros

La arquitectura se define inicialmente a través de sus registros, que se clasifican en generales, de índices, de pila, de instrucción y de control.

- **Registros Generales:** Intel utiliza registros con capacidades que han evolucionado de los 8 a los 64 bits. Entre los principales se encuentran el *Acumulador* (RAX) para operaciones aritméticas, la *Base* (RBX) para direccionamiento, el *Contador* (RCX) para bucles o cadenas, y *Data* (RDX) para duplas de registros. También existen registros numerados del R8 al R15, que mantienen la misma estructura de subdivisión de bits.

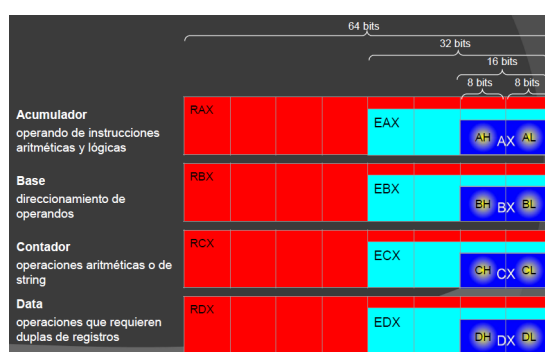


Figura 18: Algunos Registros de la Arquitectura Intel 8086

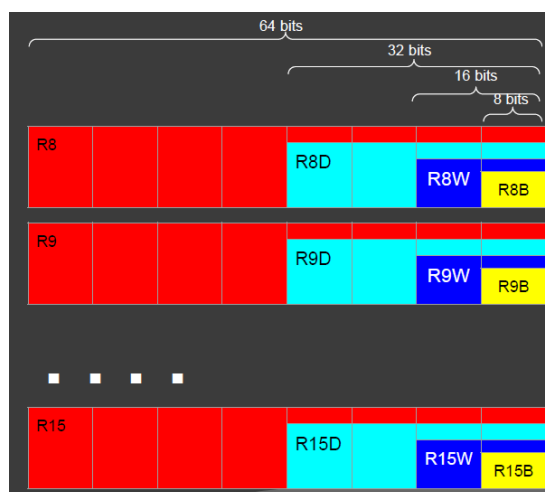


Figura 19: ¡Más Registros!

- **Registros de Índice y Pila:** Los registros RSI (*Source*) y RDI (*Destination*) se encargan del manejo de cadenas apuntando a los operandos origen y destino. Para el manejo de la memoria, se utilizan el RBP (*Base Pointer*), que marca la base de la pila, y el RSP (*Stack Pointer*), que apunta al tope actual de la misma.

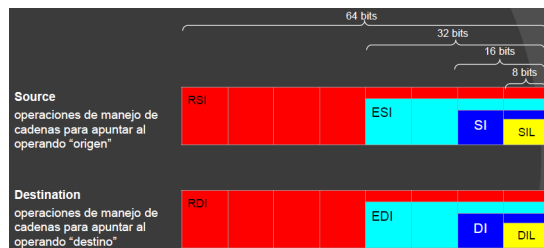


Figura 20: Registros RSI y RDI en sus Distintos Tamaños

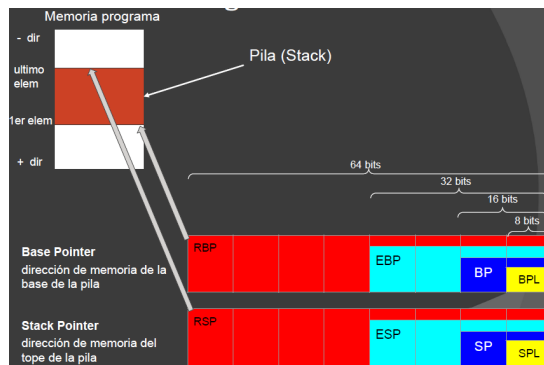


Figura 21: Registros RBP y RSP y sus Distintos Tamaños

4.7.2. Direcccionamiento y Tipos de Datos

La forma en que el procesador accede a los datos varía según el Modo de Direcccionamiento: puede ser *implícito* (en el código de operación), por *registro*, *inmediato* (el dato está en la instrucción), *directo* (referenciado por nombre), *indirecto* (vía registro) o *relativo* (usando desplazamientos y combinaciones de base e índice)

En código:

- Implícito: `cbw ;(convierte un byte en AL a una palabra en AX).`
- Registro: `ADD RAX,R14`
- Inmediato: `MOV RAX,5`
- Directo: `MOV RAX, [variable]`
- Registro Indirecto: `MOV EAX,[EBX]`
- Registro Relativo: `MOV RAX,[RBX+4]`
- Base + Índice: `MOV [RBX+RDI],CL`
- Base Relativo + Índice: `MOV RAX,[RBX+RDI+4]`

En cuanto a los datos, se manejan formatos **numéricos enteros** (punto fijo con signo), **decimales** (punto flotante IEEE 754) y **caracteres ASCII**. La memoria se organiza en celdas de 1 byte, **palabras** (2 bytes), **dobles palabras** (4 bytes) y **cuadruples palabras** (8 bytes).

4.7.3. Ensamblador NASM (Netwide Assembler)

El proceso de programación se apoya en **directivas** o pseudo-instrucciones que guían al ensamblador, pero no se traducen a código máquina, como **section**, **global**, **extern** y las definiciones de variables (**db**, **dw**, **resb**, etc.).

section	indica el comienzo de un segmento
global	indica que una etiqueta declarada en el programa es visible para un programa externo
extern	indica que una etiqueta usada en el programa pertenece a un programa externo (donde habrá sido declarada global)
db, dw, dd, dq, dt	sirven para definir áreas de memoria (variables) con contenido inicial
resb, resw, resd, resq, rest	sirven para definir áreas de memoria (variables) sin contenido inicial
times	repite una definición la cantidad de veces que se indica
%macro %endmacro	indican el inicio y final de un bloque para definir una macro
%include	permite incluir el contenido de un archivo

Figura 22: Algunas Directivas y Sus Usos

Un programa típico en NASM tiene una **estructura definida por secciones**: **.data** para variables con contenido inicial, **.bss** para variables sin contenido inicial y **.text** donde residen las instrucciones y la etiqueta **main**. La reserva de memoria puede hacerse definiendo el contenido (ej. **variable db 11**) o simplemente reservando el espacio (ej. **unByte resb 1**). Incluso se pueden definir vectores repitiendo elementos con la directiva **times**.

```
1  global main
2
3  section .data
4
5  section .bss
6
7  section .text
8  main:
9
10     ret
```

Listing 3: Estructura Elemental de un Programa ASM-x86

Para optimizar el código, se utilizan **Macros**, que son secuencias de instrucciones asignadas a un nombre que pueden recibir parámetros, y la directiva **%include** para insertar contenido de otros archivos.

4.8. Lenguaje C Embebido en Assembly y Repositorio con Código

Link: <https://github.com/ignacioagliani/Organizacion-Del-Computador>

4.9. Lenguaje Ensamblador ARM

4.9.1. Estructura Básica de un Programa

```
1      .data
2
3      cadena:      .asciz      "linea"
4      entero:      .word      78
5
6      .text
7
8      @Comentario
9      .global _start
10
11 _start:
12     swi      0x11
13     .end
```

Listing 4: Programa de Ejemplo en Assembler ARM

4.9.2. Registros Usables

- Registros Generales: $R_n, n \in \mathbb{I}_{12}$
- $R_{13 \leq n \leq 15}$, registros específicos:
 - $R_{13} \rightarrow$ Stack Pointer (SP).
 - $R_{14} \rightarrow$ Link Register (LR).
 - $R_{15} \rightarrow$ Program Counter (PC).

4.9.3. Tabla de Interrupciones de Software

Link (página 22): <https://drive.google.com/file/d/1PSvgTpCunvk4UKF-7VZOUGR1F19GFWJZ/view>

4.9.4. Algunas Instrucciones

```
1      MOV      R0, #1                ; Move
2      MOV      R1, R0
3
4      LDR      R2, =nombreVariable   ; Load Register
5      LDR      R3, [R0]              ; con '=' se cargan direcciones
6                                      ; con [] contenidos
7      STR      R4, [R0]              ; Store Register
8
9      ADD      R0, R1, R2
10     MUL      R1, R2, R3
11     AND      R_d, R_I, R_II         ; R_d -> Reg. Destino
12     ORR      R_d, R_I, R_II         ; R_I -> 1er. Operando, R_II -> 2do
```



```
EOR    R_d , R_I , R_II      ; XOR
```

Listing 5: Instrucciones Varias

4.9.5. Barrel Shifter

Los *Barrel Shifter* son una *serie de operaciones de bits* que el lenguaje ensamblador de ARM nos brinda. Esta nos permite llevar a cabo shifts como parte de otras instrucciones, **desplazando bits** según se indique.

Dependiendo de la operación, funciona de distintas maneras, por ejemplo:

- Logical Shift Left: `MOV R0, R1, LSL #2` → Deja en R0 el resultado de desplazar 2 bits a la izquierda a lo que está en R1. Es una **forma rápida de multiplicar por potencias de 2**.
- Logical Shift Right: `MOV R0, R1, LSR #3` → Deja en R0 el resultado de desplazar 3 bits a la derecha el contenido de R1. Es una **forma rápida de dividir por potencia de 2**.
- Arithmetic Shift Right: `MOV R0, R1, ASR #4` → Igual que LSR, pero mantiene el signo.

4.9.6. Ejecucion Condicional

En ensamblador ARM, la **ejecución condicional** es una característica por la cual casi cualquier instrucción puede ejecutarse solo si se cumple una condición específica, en lugar de depender únicamente de bifurcaciones (saltos) como en otras arquitecturas.

La instrucción es ejecutada sólo si el estado actual del flag del código de condición del procesador satisface la condición especificada en los bits 31 a 28 de la instrucción.

Ejemplo: Deja en R0 la suma de R1 con R2 si el *flag cero está seteado*:

```
ADDEQ r0, r1, r2
```

Normalmente antes de una instrucción condicional va un `CMP`.

4.9.7. Instrucción Branch

Similar a los `JMP` de ASM-x86. Muy usado para loops. Código Auto-Explicativo:

```
1  mov    R0, #1
2  b      algun_lugar
3  mov    R0, #5
4
5  algun_lugar:
6  swi    SWI_EXIT
```

Listing 6: Branch Incondicional

```

1      mov     R0, #1
2      mov     R1, #5
3      cmp     R1, #5
4      beq     es_equal
5
6 es_equal:
7      swi     SWI_EXIT

```

Listing 7: Branch Condicional

4.9.8. Iterar Vectores

```

1      .data
2
3      vector:      .word      32, 65, 76
4
5      .text
6
7      .global _start
8
9 _start:
10     ldr     r0, =vector
11     ldr     r1, [r0]
12     add     r0, r0, #4      @ Avanzo de a 4 porque sizeof(word) = 4
13     ldr     r2, [r0]
14     add     r0, r0, #4
15     ldr     r3, [r0]
16     swi     0x11
17     .end

```

Listing 8: Iteración de Vector en Ensamblador de ARM

(Hay más código en el repo de GitHub).

4.9.9. Modos de Direccionamiento

Nombre	Nombre Alternativo	Ejemplos
Registro a registro	Registro directo	<code>mov r0, r1</code>
Absoluto	Directo	<code>ldr r0, mem</code>
Literal	Inmediato	<code>mov r0, #15</code> <code>add r1, r2, #12</code>
Indexado, base	Registro indirecto	<code>ldr r0, [r1]</code>
Pre-Indexado, base con desplazamiento	Registro indirecto con offset	<code>ldr r0, [r1, #4]</code>
Pre-indexado, autoindexado	Registro indirecto con pre-incremento	<code>ldr r0, [r1, #4]!</code>
Post-indexado, autoindexado	Registro indirecto con post-incremento	<code>ldr r0, [r1], #4</code>
Doble registro indirecto	Registro indirecto indexado	<code>ldr r0, [r1, r2]</code>
Doble registro indirecto escalado	Registro indirecto indexado escalado	<code>ldr r0, [r1, r2, lsl #2]</code>
Relativo al PC		<code>ldr r0, [PC, #offset]</code>

1. Indexado, base (Registro indirecto)

- **Sintaxis:** `ldr r0, [r1]`
- **Cómo funciona:** Accede directamente a la dirección de memoria apuntada por el valor actual de `r1`.
- **Resultado:** Copia el dato de esa dirección en `r0`. El valor de `r1` no cambia.

2. Pre-Indexado, base con desplazamiento (Registro indirecto con offset)

- **Sintaxis:** `ldr r0, [r1, #4]`
- **Cómo funciona:** Calcula una dirección temporal sumando el registro base `r1` más un desplazamiento fijo (en este caso, 4 bytes). Lee el dato de esa nueva posición simulada.
- **Resultado:** Guarda el dato en `r0`. `r1` mantiene su valor original (no se actualiza en memoria).

3. Pre-indexado, autoindexado (Registro indirecto con pre-incremento)

- **Sintaxis:** `ldr r0, [r1, #4]!`
- **Cómo funciona:** El signo de exclamación activa el escribir de vuelta (*write-back*). El procesador primero suma 4 a `r1`, luego lee el dato de esa nueva dirección calculada y lo guarda en `r0`.
- **Resultado:** Copia el dato en `r0` y además modifica permanentemente a `r1` ($r1 = r1 + 4$).

4. Post-indexado, autoindexado (Registro indirecto con post-incremento)

- **Sintaxis:** `ldr r0, [r1], #4`
- **Cómo funciona:** Aquí el desplazamiento `#4` está fuera de los corchetes. El procesador primero lee el dato de la dirección exacta que tiene `r1` en ese instante y lo guarda en `r0`. Después de hacer la lectura, le suma 4 a `r1`.
- **Resultado:** Copia el dato original en `r0` y actualiza el puntero ($r1 = r1 + 4$). Ideal para recorrer arreglos en bucles.

5. Doble registro indirecto (Registro indirecto indexado)

- **Sintaxis:** `ldr r0, [r1, r2]`
- **Cómo funciona:** En lugar de usar un número fijo (`#4`), usa el valor almacenado en otro registro (`r2`) como el desplazamiento. La dirección destino es la suma matemática de $r1 + r2$.
- **Resultado:** Copia el dato de esa dirección combinada en `r0`. Ningún registro base cambia.

6. Doble registro indirecto escalado (Registro indirecto indexado escalado)

- **Sintaxis:** `ldr r0, [r1, r2, lsl #2]`
- **Cómo funciona:** Permite multiplicar el índice sobre la marcha usando el desplazador de barril (*Barrel Shifter*). `lsl #2` significa “desplazamiento lógico a la izquierda por 2 bits”, lo que equivale matemáticamente a multiplicar `r2` por 4 (2^2). La dirección final es $r1 + (r2 \cdot 4)$.
- **Resultado:** Es la forma nativa de ARM para acceder a un índice específico de un arreglo de enteros (donde cada elemento mide 4 bytes).

7. Relativo al PC

- **Sintaxis:** `ldr r0, [PC, #offset]`
- **Cómo funciona:** El PC (*Program Counter*) apunta a la instrucción que se está ejecutando en ese momento. Este modo calcula una dirección sumando o restando una distancia (*offset*) respecto a la posición actual del código en la memoria.
- **Resultado:** Se utiliza principalmente para cargar constantes o variables globales que están guardadas muy cerca del propio código fuente (literales en memoria).

5. Unidad 5: Componentes de un Computador

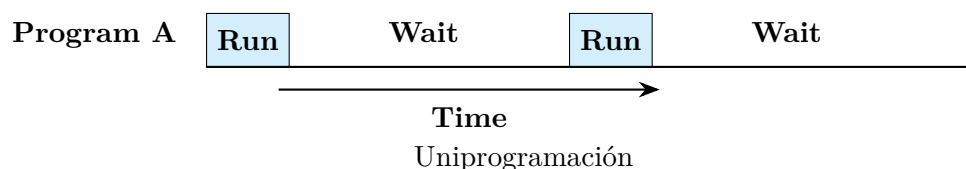
5.1. Administración de Memoria

5.1.1. El Sistema Operativo como gestor de recursos

Para comprender la administración de memoria, es necesario partir del concepto de **Sistema Operativo**, entendido como el *software que administra los recursos del computador, provee servicios y controla la ejecución de otros programas*. Entre los servicios que provee se encuentran el scheduling de procesos y, precisamente, la administración de memoria. Dentro del Sistema Operativo existe una componente especial denominada **Monitor**, que constituye su parte residente en memoria, es decir, aquella que permanece cargada en todo momento durante la operación del sistema.

5.1.2. Uniprogramación

El modelo más simple de ejecución es la *uniprogramación*, en la cual existe **un único proceso de usuario en ejecución a la vez**. En este esquema, la memoria de usuario está completamente disponible para ese único proceso. Al analizar el uso del procesador a lo largo del tiempo, se distinguen dos estados: el tiempo *Run*, que representa el uso efectivo del CPU, y el tiempo *Wait*, que corresponde al tiempo ocioso del procesador mientras espera operaciones de entrada/salida, también conocido como idle time.



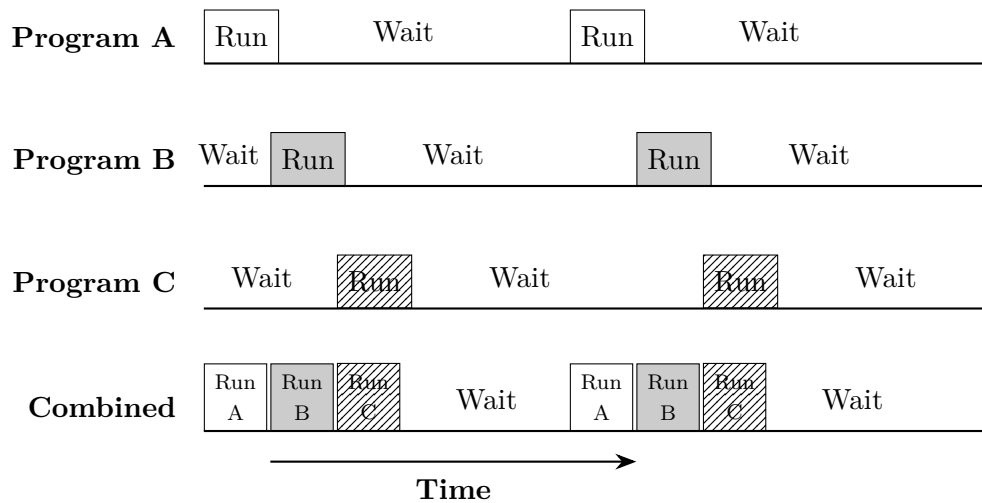
5.1.3. Administración de memoria simple

La *administración de memoria simple* es la estrategia asociada a los sistemas con **uniprogramación**. En ella, la memoria se divide en dos partes: una reservada para el **Monitor** del Sistema Operativo y otra destinada al **programa en ejecución** en ese momento. Su principal ventaja es la **simplicidad**. Sin embargo, presenta desventajas importantes: el **desperdicio de memoria** y el **desaprovechamiento general de los recursos** del computador. Ejemplos históricos de este modelo son MS-DOS, las primeras versiones del iPhone OS (v1 a v3) y el IBM OS/PCP (Primary Control Program).

5.1.4. Multiprogramación

Para superar las limitaciones de la uniprogramación surge la *multiprogramación*, modelo en el que **varios procesos de usuario se encuentran en ejecución de manera simultánea**. En este esquema, la memoria de usuario se divide entre los distintos procesos activos, y el tiempo del procesador se comparte entre ellos mediante un mecanismo denominado *timeslice*. Existen cuatro condiciones bajo las cuales un **proceso puede dejar de**

ocupar el procesador: cuando termina su trabajo, cuando se detecta un error y es cancelado, cuando requiere una operación de entrada/salida —lo que provoca su suspensión—, y cuando finaliza su timeslice asignado, también resultando en una suspensión.



(c) Multiprogramming with three programs

5.1.5. Administración de memoria por asignación particionada

En sistemas con multiprogramación, una primera aproximación es la *administración de memoria por asignación particionada*, en la que la **memoria de usuario se divide en particiones de tamaño fijo**. Estas particiones pueden ser de tamaño igual entre sí o de tamaños distintos, dependiendo de la configuración del sistema. La ventaja principal de este esquema es que permite **compartir la memoria entre varios procesos** de manera simultánea. No obstante, su desventaja central es el **desperdicio de memoria**, que se manifiesta de dos formas: la *fragmentación interna*, que ocurre dentro de una partición cuando el proceso no la ocupa completamente, y la *fragmentación externa*, que se produce cuando existen particiones disponibles pero ningún proceso puede utilizarlas. El ejemplo más representativo de este modelo es el IBM OS/MFT (Multiprogramming with a Fixed number of Tasks).

5.1.6. Administración de memoria paginada simple

La *administración de memoria paginada simple* introduce un mecanismo más sofisticado para los sistemas con multiprogramación. El espacio de direcciones de cada proceso se divide en partes iguales llamadas *páginas* —en arquitecturas IA-32, por ejemplo, cada página tiene 4 KB—, mientras que la memoria principal se divide en partes del mismo tamaño denominadas *frames*. Por cada proceso existe una tabla de páginas, y el sistema mantiene una lista global de frames disponibles. Al cargar un proceso, sus páginas se alojan en los frames libres sin que sea necesario que sean contiguos en memoria física. Las direcciones lógicas se expresan como un número de página más un desplazamiento u offset, y son traducidas a direcciones físicas mediante un mecanismo de address translation soportado por hardware, específicamente la MMU o Memory Management Unit. Un aspecto clave de este modelo es que la paginación es completamente transparente para el programador. Entre sus ventajas se destacan la posibilidad de **compartir memoria**

entre procesos, el uso no contiguo de la memoria, la **minimización de la fragmentación interna** —que solo puede existir dentro de la última página de cada proceso— y la **eliminación total de la fragmentación externa**. Sus desventajas son la necesidad de **cargar todas las páginas del proceso en memoria** antes de su ejecución, y el **requerimiento de estructuras de datos** adicionales para administrar la información de páginas y frames.

5.1.7. Administración de memoria paginada por demanda (memoria virtual)

El paso siguiente en evolución es la *administración de memoria paginada por demanda*, también conocida como memoria virtual. A diferencia de la paginación simple, aquí solo **se cargan en memoria principal las páginas estrictamente necesarias** para la ejecución del proceso en cada momento. Cuando se intenta acceder a una posición de memoria cuya página no está cargada, se produce un *page fault*. Este evento dispara una interrupción por hardware generada por la MMU, que es atendida por el Sistema Operativo. El componente del S.O. encargado de resolverlo, el page fault handler, recupera la página solicitada desde la memoria secundaria —el almacenamiento en disco que actúa como memoria virtual. Si no hay frames libres disponibles, el sistema debe bajar páginas a memoria secundaria para liberar espacio, en un proceso denominado *page swapping*. Para decidir qué páginas desalojar, se aplican algoritmos de reemplazo como FIFO (First In First Out) o LRU (Least Recently Used). Un fenómeno negativo que puede surgir en este contexto es el **thrashing**, situación en la que *el CPU dedica más tiempo a reemplazar páginas que a ejecutar instrucciones útiles*. Las ventajas de este modelo incluyen que **no es necesario cargar todas las páginas** de un proceso a la vez, que se **maximiza el uso de la memoria** al permitir cargar más procesos simultáneamente, y que un proceso puede incluso **ocupar más memoria virtual de la físicamente instalada** en el sistema. Su desventaja es la **mayor complejidad** que implica implementar el mecanismo de reemplazo de páginas. Sistemas operativos como Windows 3.x en adelante y Linux son ejemplos de este enfoque.

5.1.8. Administración de memoria por segmentación

La *segmentación* es otra estrategia de administración de memoria para sistemas con multiprogramación, y a diferencia de la paginación, generalmente es visible al programador. En este modelo, la memoria de un programa se concibe como un conjunto de segmentos, es decir, múltiples espacios de direcciones diferenciados, cuyos **tamaños son variables y dinámicos** según las necesidades del programa. El Sistema Operativo mantiene una *tabla de segmentos* por cada proceso. Este esquema presenta ventajas funcionales importantes: permite separar datos e instrucciones en segmentos distintos, y habilita la aplicación de privilegios y protección de memoria, como permisos de lectura, escritura y ejecución. Los accesos indebidos generan *segmentation faults*, que son mecanismos de excepción de hardware. Las referencias a memoria se construyen con un número de segmento más un offset dentro de él, y la traducción de direcciones lógicas a físicas también cuenta con el apoyo de la MMU. Adicionalmente, la segmentación puede utilizarse para implementar memoria virtual, cargando en memoria física únicamente algunos segmentos de cada proceso. Entre sus ventajas se destacan la **simplificación del manejo de estructuras de datos** con crecimiento dinámico, la posibilidad de **compartir información entre procesos** dentro de un mismo segmento, y la **facilidad para aplicar protección**

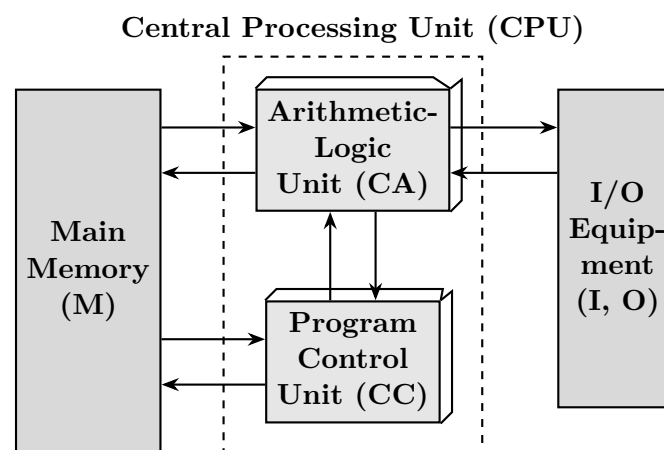
y privilegios por segmento. Sus desventajas son la **fragmentación externa en memoria principal** —cuando no hay espacio contiguo para alojar un segmento— y la **mayor complejidad del hardware** necesario para la traducción de direcciones en comparación con la paginación. Ejemplos históricos incluyen la Burroughs Corporation B5000-B6500, el IBM AS/400 y la arquitectura Intel x86, que la mantiene por compatibilidad hacia atrás.

5.1.9. Combinaciones de segmentación y paginación

Los mecanismos de paginación y segmentación no son excluyentes; pueden combinarse de distintas maneras, como ocurre por ejemplo en la arquitectura Intel Pentium. Existen cuatro variantes posibles. La primera es operar **sin segmentación y sin paginación**, situación en la que las direcciones lógicas coinciden con las físicas; este modo no es útil para multiprogramación y se emplea en controladores de alta performance. La segunda es la **paginación sin segmentación**, donde la protección y administración de la memoria se realiza a través de las páginas, como en Berkeley UNIX. La tercera es la **segmentación sin paginación**, en la que la memoria se presenta como una colección de espacios lógicos con protección a nivel de segmento. La cuarta y más completa combinación es la **segmentación con paginación**, donde los segmentos controlan el acceso a particiones de memoria mientras que las páginas administran la localización dentro de cada segmento; UNIX System V es el ejemplo representativo de este enfoque.

5.2. Entrada/Salida

En la arquitectura de un computador, la CPU y la memoria principal no operan de forma aislada: necesitan comunicarse con una amplia variedad de dispositivos periféricos. Esta comunicación se materializa a través del Módulo de E/S, que actúa como intermediario entre el bus del sistema —o switch central— y los equipos de entrada y salida.



Para comprender su razón de ser, conviene responder tres preguntas fundamentales. En primer lugar, ¿qué hace el módulo de E/S? Su función esencial es conectar los periféricos con la CPU y la memoria, posibilitando la comunicación entre estos componentes. En segundo lugar, ¿por qué existe? La respuesta radica en tres factores: la enorme variedad de periféricos disponibles, cada uno con sus propios métodos de operación; el hecho de que la tasa de transferencia de los periféricos es generalmente mucho más lenta

que la de la memoria y el procesador; y que los distintos dispositivos utilizan formatos de datos y tamaños de palabra heterogéneos. Finalmente, **¿para qué sirve concretamente?** El módulo de E/S oculta todos los detalles de timing, formatos y electromecánica de los dispositivos periféricos, presentando una interfaz uniforme al resto del sistema.

5.2.1. Interfaces del módulo de E/S

El módulo de E/S posee *dos interfaces* bien diferenciadas. Hacia el interior del sistema, la **interfaz interna** se conecta al bus del sistema y maneja tres tipos de señales: líneas de datos, líneas de direcciones y líneas de control. Hacia el exterior, la **interfaz externa** se comunica con cada periférico a través de señales de datos, señales de control y señales de estado, estas últimas permitiendo conocer la condición operativa del dispositivo en cada momento.

5.2.2. Funciones del módulo de E/S

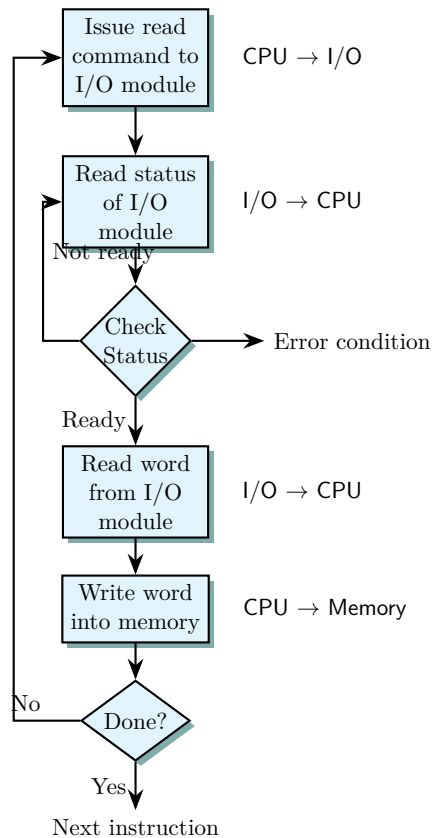
El módulo de E/S desempeña un conjunto de funciones que pueden agruparse en cinco categorías. La primera es el **Control & Timing**, mediante el cual regula y sincroniza el flujo de tráfico entre la CPU, la memoria y los periféricos. La segunda es la **comunicación con el procesador**, que comprende la decodificación de comandos recibidos desde la CPU, el intercambio de datos, el reporte de información de estado y el reconocimiento de las direcciones de los dispositivos bajo su control. La tercera es la **comunicación con el dispositivo**, que involucra el envío de comandos al periférico, la recepción de información de estado y la transferencia de datos en ambas direcciones. La cuarta es el **buffering de datos**, que permite compensar la diferencia de velocidad entre el periférico y el bus del sistema almacenando temporalmente la información en tránsito. La quinta y última es la **detección de errores**, por la cual el **módulo identifica y reporta condiciones anómalas** que puedan surgir durante las transferencias.

5.2.3. Técnicas para operaciones de E/S

Existen tres técnicas fundamentales mediante las cuales el sistema puede gestionar las operaciones de E/S, cada una con distintos niveles de participación del procesador.

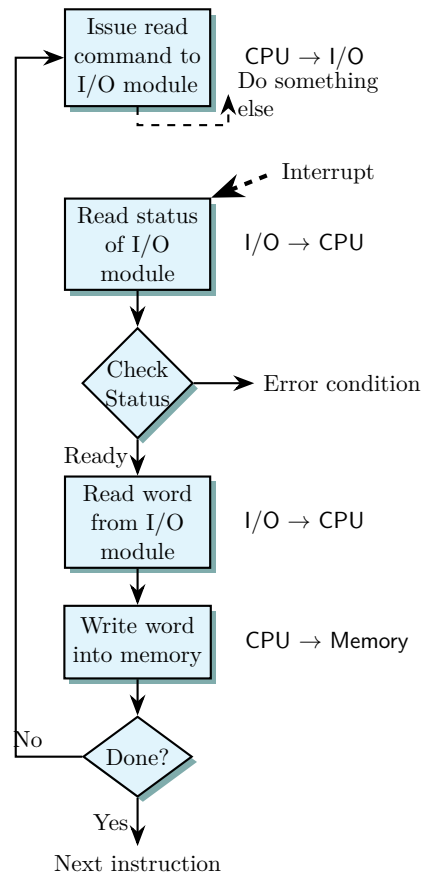
5.2.4. E/S Programada

La E/S programada es la técnica más directa y también la más costosa en términos de utilización del procesador. En este esquema, la CPU emite un comando de lectura al módulo de E/S y luego queda ocupada verificando repetidamente el estado del módulo — un proceso conocido como *polling* o *espera activa*— hasta que la operación esté completa. Una vez que el módulo reporta estar listo, la CPU lee el dato desde el módulo y lo escribe en memoria. Este ciclo se repite hasta que la transferencia completa haya finalizado. La desventaja principal es evidente: el **procesador desperdicia ciclos en la espera**, sin poder ejecutar otras tareas.



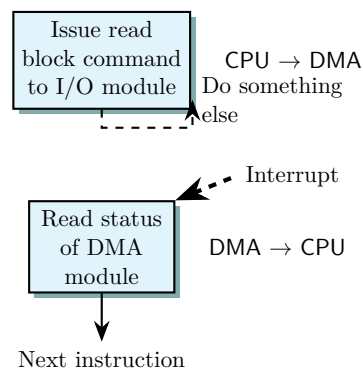
5.2.5. E/S manejada por interrupciones

La E/S manejada por interrupciones mejora significativamente la eficiencia del procesador. En este caso, la CPU emite el comando de lectura al módulo de E/S y, en lugar de quedarse esperando, continúa ejecutando otras instrucciones. Cuando el módulo de E/S ha completado su operación y tiene el dato disponible, envía una **señal de interrupción a la CPU**. Esta interrupción hace que el procesador pause su tarea actual, lea el dato desde el módulo y lo escriba en memoria, y luego retome lo que estaba haciendo. De esta manera, el procesador aprovecha el tiempo que el periférico tarda en responder para realizar trabajo útil.



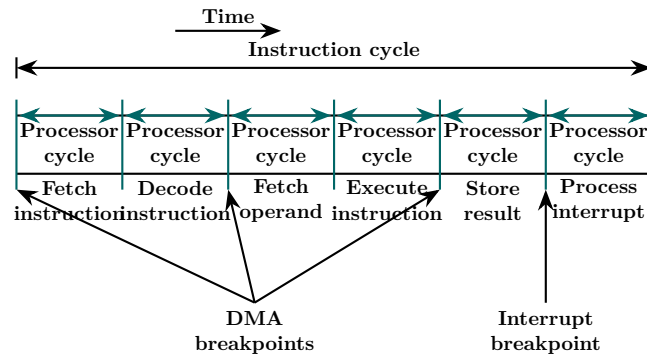
5.2.6. Acceso directo a memoria (DMA)

El DMA representa la técnica más avanzada y eficiente para grandes transferencias de datos. En este esquema, la **CPU delega la gestión completa de la transferencia a un módulo DMA** dedicado. Para iniciar la operación, la CPU envía al DMA la siguiente información: el tipo de operación a realizar —READ o WRITE— a través de la línea de control; la *dirección del dispositivo* involucrado a través de la línea de dirección; la *dirección inicial de memoria* donde se leerán o escribirán los datos, transmitida por la línea de datos y almacenada en el address register del DMA; y la *cantidad de palabras a transferir*, también por la línea de datos y almacenada en el data count register.

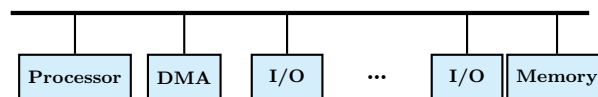


Una vez configurado, el DMA toma control del bus y realiza la transferencia de manera

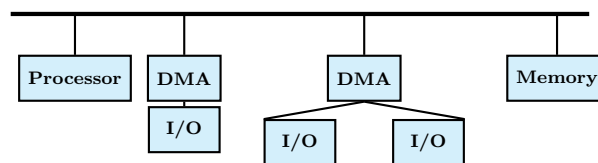
autónoma, mientras la CPU puede dedicarse a otras tareas. Cuando la transferencia concluye, el **DMA notifica a la CPU mediante una interrupción**. Este mecanismo requiere acceder al bus del sistema en momentos en que el procesador no lo está utilizando, lo que se conoce como “robo de ciclos”: el DMA aprovecha los instantes entre ciclos del procesador —como entre el fetch del operando y la ejecución de la instrucción, o al almacenar el resultado— para transferir datos sin interferir con la ejecución normal del programa.



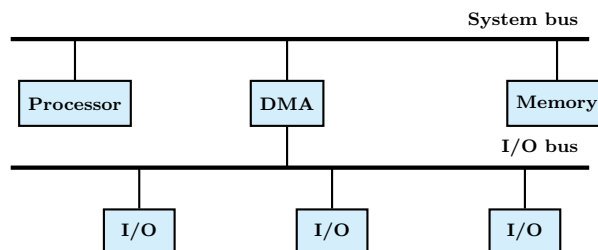
En cuanto a las topologías de configuración del DMA, existen tres variantes principales. La primera es el **single-bus con DMA separado**, donde el módulo DMA y los módulos de E/S comparten el mismo bus del sistema junto con el procesador y la memoria. La segunda es el **single-bus con DMA e E/S integrados**, donde el DMA incorpora directamente la lógica de control de los módulos de E/S. La tercera es la configuración con **bus de E/S independiente**, donde el DMA se comunica con los módulos de E/S a través de un bus dedicado separado del bus del sistema principal.



(a) Single-bus, detached DMA



(b) Single-bus, integrated DMA-I/O



(c) I/O bus

5.2.7. Canales y procesadores de E/S

Una evolución adicional del concepto de módulo de E/S la constituyen los *canales* y *procesadores* de E/S, orientados a sistemas que requieren una gestión aún **más autónoma** de las operaciones de entrada/salida.

Los **canales** de E/S tienen la capacidad de *ejecutar instrucciones de E/S de manera independiente*, de modo que la CPU principal no necesita involucrarse en su ejecución. Las instrucciones de E/S se almacenan en la memoria principal, y la CPU simplemente le indica al canal que inicie un programa de canal, especificando cuatro parámetros: el *dispositivo* con el que se debe operar, el área de *memoria destinada* al almacenamiento de datos, la *prioridad* de la operación y las *acciones a tomar ante eventuales errores*. El canal se encarga del resto de manera autónoma.

Los **procesadores** de E/S van un paso más allá en esta evolución: a diferencia de los canales, que utilizan la memoria principal para almacenar sus instrucciones, los procesadores de E/S incorporan memoria propia, ganando así mayor independencia y capacidad de procesamiento.

En cuanto a los **tipos de canales**, se distinguen dos variantes. Los *canales selectores* operan con un único dispositivo a la vez, a través de un controlador de E/S, priorizando la velocidad de transferencia hacia ese dispositivo. Los *canales multiplexores*, en cambio, pueden trabajar con múltiples dispositivos simultáneamente, multiplexando los flujos de datos y permitiendo así mayor concurrencia en las operaciones de E/S.

5.3. Interrupciones

5.3.1. ¿Qué son las interrupciones y para qué existen?

Las *interrupciones* son **mecanismos** por los cuales otros módulos del sistema —como los de entrada/salida, la memoria u otros componentes— interrumpen el procesamiento normal del CPU. Su razón de existir es concreta y fundamental: **mejorar la eficiencia de procesamiento** del computador. Sin este mecanismo, el procesador debería dedicar tiempo a esperar activamente que los dispositivos respondan, desperdiciando ciclos que podrían emplearse en trabajo útil.

Las interrupciones se clasifican en **dos grandes categorías**: las de *hardware* y las de *software*, cada una con características y orígenes distintos.

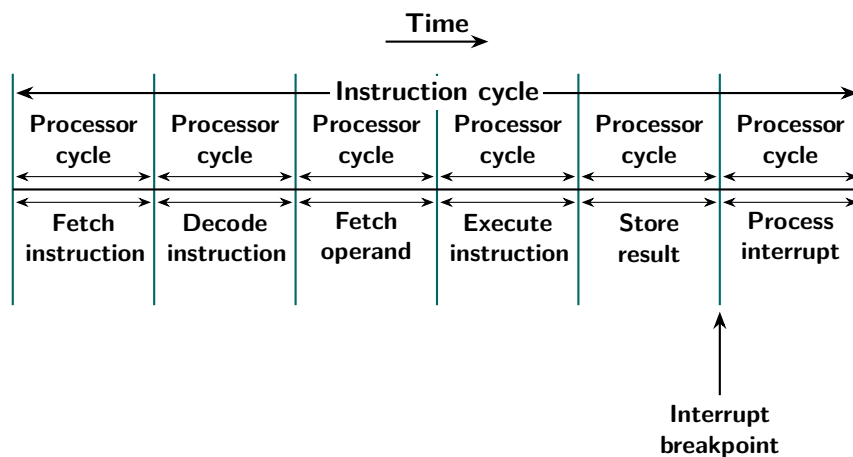
5.3.2. Clases de interrupciones

Las interrupciones de **hardware** son asincrónicas, es decir, pueden producirse en cualquier momento, sin relación directa con el flujo de instrucciones que ejecuta el procesador. Dentro de esta categoría se distinguen **tres tipos**: las interrupciones generadas por *dispositivos de entrada/salida*, que señalan la finalización de una operación o la disponibilidad de datos; las de *reloj* o timer, utilizadas por el sistema operativo para implementar el scheduling de procesos mediante timeslices; y las provocadas por fallas de *hardware*, como errores de paridad en memoria u otras condiciones anómalas del sistema físico.

Las interrupciones de **software**, en cambio, tienen su origen en la ejecución misma del programa y se dividen en dos subtipos. El primero son las *excepciones de programa*, que se producen cuando una instrucción genera una condición de error, como una división por cero, un acceso indebido a una posición de memoria protegida, un desbordamiento aritmético u overflow, o el intento de ejecutar una instrucción inválida. El segundo subtipo son las *instrucciones privilegiadas*, que corresponden a llamadas explícitas realizadas por el programa para solicitar servicios al sistema operativo, y que en arquitecturas modernas actúan como una forma controlada de transferir el control al núcleo del sistema.

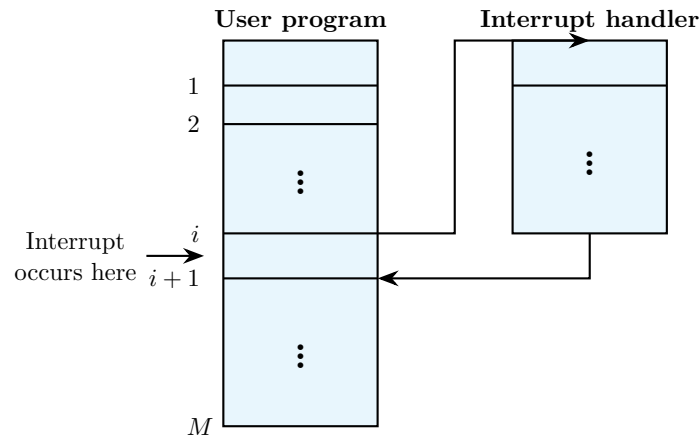
5.3.3. Las interrupciones en el ciclo de instrucción

Para comprender *cómo se integran* las interrupciones al funcionamiento del procesador, es necesario ubicarlas dentro del *ciclo de instrucción*. Este ciclo se compone de **varias etapas** que se ejecutan secuencialmente en cada ciclo de procesador: fetch de la instrucción, decodificación, fetch del operando, ejecución, almacenamiento del resultado y, finalmente, procesamiento de la interrupción. Esta última etapa es precisamente el punto donde el procesador verifica si existe alguna interrupción pendiente. Si la hay, el procesador la atiende antes de iniciar el ciclo siguiente; si no, continúa con normalidad con la próxima instrucción.



5.3.4. Transferencia de control al handler

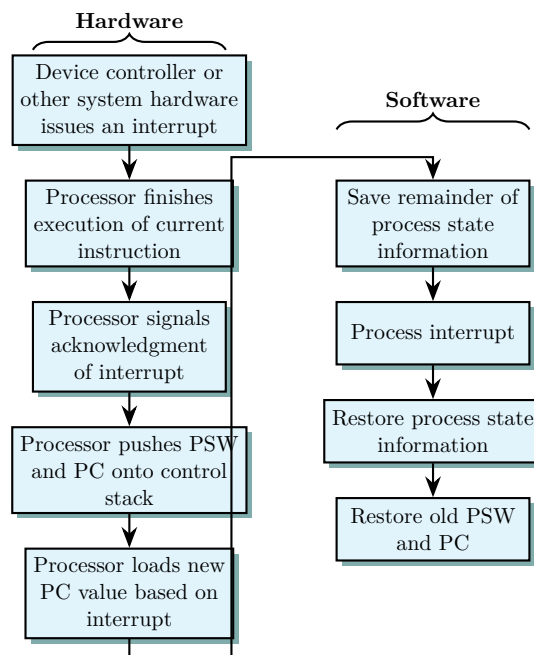
Cuando se produce una interrupción durante la ejecución del programa de usuario, el control se transfiere al sistema operativo, específicamente a una rutina de atención denominada *interrupt handler*. El programa de usuario queda detenido en el punto exacto donde ocurrió la interrupción —la instrucción i — y, una vez que el handler concluye su tarea, el control retorna al programa en la instrucción siguiente, es decir, en $i + 1$, reanudando la ejecución de manera transparente para el usuario.



5.3.5. Procesamiento de una interrupción

El *procesamiento de una interrupción* implica una **secuencia coordinada de acciones** que involucran tanto al hardware como al software del sistema. Del lado del hardware, el proceso comienza cuando un controlador de dispositivo u otro componente del sistema emite la señal de interrupción. El procesador finaliza la ejecución de la instrucción en curso —sin interrumpirla a mitad— y luego señaliza al dispositivo que la interrupción ha sido reconocida. A continuación, el procesador guarda en el stack de control el Program Status Word (PSW) y el Program Counter (PC), que contienen el estado actual de la ejecución. Finalmente, carga en el PC el nuevo valor correspondiente a la dirección del handler de la interrupción, transfiriendo así el control al software.

Del lado del software, el handler comienza guardando el estado restante del proceso interrumpido —los registros de propósito general y demás información de contexto—. Luego procesa la interrupción realizando las acciones correspondientes según su naturaleza. Una vez concluida la atención, restaura la información de estado previamente guardada y, finalmente, recupera el PSW y el PC originales para que el procesador retome la ejecución del programa interrumpido exactamente desde donde se detuvo.



5.3.6. Múltiples interrupciones

En un sistema real, pueden producirse **varias interrupciones de manera simultánea** o solapada. Para gestionar esta situación existen dos estrategias fundamentales.

La primera consiste en deshabilitar las interrupciones durante el procesamiento de una interrupción activa, lo que da lugar a un *procesamiento secuencial*. Bajo este esquema, si mientras se atiende la interrupción $\mathcal{X}$ se produce la interrupción $\mathcal{Y}$, esta última **queda pendiente** y será atendida recién cuando el handler de $\mathcal{X}$ haya terminado y las interrupciones vuelvan a habilitarse. El resultado es una secuencia estrictamente lineal de atención. La **ventaja** de esta estrategia es la simplicidad en el diseño del hardware y del sistema operativo y la **desventaja** es que no tiene en cuenta que hay interrupciones con más prioridad que otras.

La segunda estrategia consiste en priorizar las interrupciones y permitir el procesamiento anidado. En este caso, si mientras se está atendiendo una interrupción de menor prioridad llega una de mayor prioridad, el handler en ejecución **puede ser interrumpido** para atender primero la nueva interrupción, y luego retomar la anterior. Este mecanismo permite una respuesta más ágil ante eventos críticos del sistema. La **ventaja** de este método es que soluciona el problema de la otra estrategia en lo que respecta a la prioridad de las interrupciones y la **desventaja** es que demanda un diseño de software y hardware más sofisticado.

5.4. Memoria

El *sistema de memoria* no es una entidad única, sino que está formado por diversos elementos que se distinguen según su tecnología, organización, performance y costo. Dentro de esta estructura, existe una **jerarquía** de subsistemas dividida en dos grandes grupos: los *internos al sistema*, que son accedidos directamente por el procesador, y los *externos*, a los cuales el procesador accede mediante un módulo de entrada/salida (E/S).

5.4.1. Jerarquía y Rendimiento

Para comprender la jerarquía de memoria, es fundamental considerar **tres características críticas**: *capacidad, tiempo de acceso y costo*. Debido a que existe una relación de compromiso (trade-off) entre estas variables, no se utiliza un solo componente de memoria en la arquitectura. Esta jerarquía se visualiza como una pirámide que incluye, desde la cúspide hacia la base: registros, memoria caché, memoria principal (almacenamiento interno), discos magnéticos y medios ópticos (almacenamiento externo), y finalmente cintas magnéticas (almacenamiento fuera de línea).

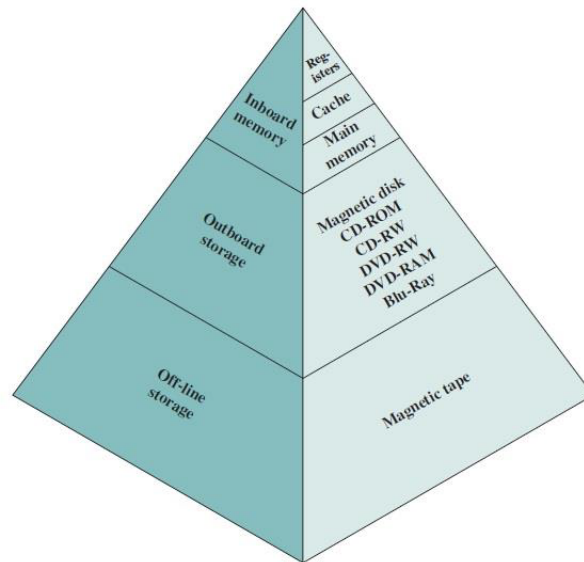


Figura 23: Jerarquía de Memoria

A medida que se desciende por esta pirámide, se observan cuatro tendencias claras: el costo por bit disminuye, la capacidad aumenta, el tiempo de acceso se vuelve más lento y la frecuencia con la que el procesador accede a dicha memoria decrece.

5.4.2. Características y Métodos de Acceso

El sistema de memoria se define por varias características distintivas:

- **Locación:** Como mencionamos al principio, puede ser *interna* (registros, memoria de la unidad de control y caché) o *externa* (dispositivos periféricos como discos y cintas).
- **Capacidad y Transferencia:** La memoria interna se mide en bytes o palabras, mientras que la externa suele medirse únicamente en bytes. La unidad de transferencia varía desde el tamaño de palabra o múltiplos (64 a 256 bits) en memorias internas, hasta bloques en memorias externas.
- **Métodos de Acceso:** Existen cuatro formas principales de acceder a los datos:
 - **Secuencial:** Acceso lineal con tiempo variable. Se deben pasar y descartar todos los registros intermedios antes de acceder al registro deseado (ej. cintas magnéticas).
 - **Directo:** Basado en la posición física con tiempo variable. Cuenta con una dirección única para cada bloque de memoria (ej. discos magnéticos).
 - **Aleatorio:** Cada posición tiene un mecanismo cableado físicamente, ofreciendo un tiempo de acceso constante (ej. memoria principal).
 - **Asociativo:** Un tipo de acceso aleatorio que busca en tiempo constante por contenido en lugar de dirección (ej. memoria caché).

En cuanto a la **performance**, se consideran parámetros como el *tiempo de acceso* (latencia), el *tiempo de ciclo de memoria* y la *tasa de transferencia*, calculada de forma distinta según si la memoria es de acceso aleatorio o no.

5.4.3. Tiempo de acceso (Latencia)

El significado de este parámetro varía según el tipo de tecnología:

- **En memorias de acceso aleatorio (RAM):** Es el tiempo exacto que tarda la memoria en realizar una operación completa de lectura o escritura desde que se recibe la dirección.
- **En memorias sin acceso aleatorio (como discos o cintas):** Es el tiempo necesario para posicionar físicamente el mecanismo de lectura/escritura en la ubicación deseada.

5.4.4. Tiempo de ciclo de memoria

Este concepto se aplica específicamente a las **memorias de acceso aleatorio**. No es solo el tiempo de acceso, sino que incluye un “tiempo adicional” necesario para que la memoria se recupere o se estabilice antes de que pueda comenzar una nueva operación.

5.4.5. Tasa de transferencia

Es la *velocidad a la que los datos entran o salen* de la unidad de memoria. Su cálculo depende estrictamente del método de acceso:

- **Para memorias de acceso aleatorio:** La tasa es simplemente la inversa del tiempo de ciclo ($\frac{1}{T_{\text{de ciclo}}}$).
- **Para memorias sin acceso aleatorio:** Se utiliza una fórmula para calcular el tiempo promedio (T_n) para leer o escribir n bits:

$$T_n = T_A + \frac{n}{R}$$

T_A : Tiempo promedio de acceso.

n : Número de bits a transferir.

R : Tasa de transferencia en bits por segundo (bps).

5.4.6. Tipos Físicos y Tecnologías de Memoria

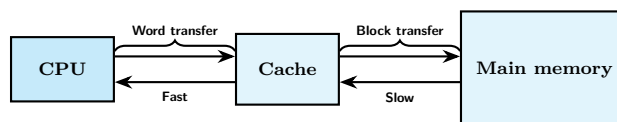
Físicamente, encontramos memorias *semiconductoras* (memoria principal y cache), de *superficie magnética* (discos y cintas) y *ópticas* (medios ópticos). Estas pueden clasificarse según su volatilidad: las **volátiles pierden su contenido** sin energía eléctrica, mientras que las **no volátiles lo mantienen**. También existen las memorias de solo lectura (ROM) cuyo contenido no se puede borrar.

Dentro de las memorias semiconductoras de acceso aleatorio, destacan:

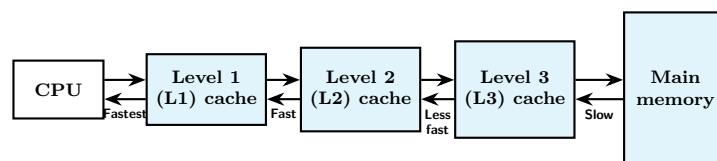
- **DRAM (Dynamic RAM):** Almacena datos como carga en capacitores y requiere refresco periódico; es común en la memoria principal.
- **SRAM (Static RAM):** Utiliza flip-flops, es más rápida y compleja que la DRAM, y se emplea usualmente en la memoria caché.
- **Tipos de DRAM:** Como la SDRAM, que se sincroniza con el reloj del sistema, y la DDR DRAM, que incrementa la tasa de transferencia mediante esquemas de *buffering*.

5.4.7. Memoria Caché

La *memoria caché* es una memoria semiconductora más **rápida y costosa** que la principal, ubicada estratégicamente entre el procesador y esta última, para **mejorar la performance general**. Su funcionamiento se basa en el *principio de localidad de referencia*, que establece que las referencias a memoria tienden a agruparse en el tiempo y el espacio.



(a) Single cache



(b) Three-level cache organization

5.4.8. ¿Cómo funciona?

Cuando el CPU intenta leer una palabra, se verifica primero en la caché. Si no está, se lee un bloque completo de la memoria principal y se incorpora a la caché, anticipando que las próximas palabras buscadas estarán en ese mismo bloque (por el principio de localidad de referencia). Arquitecturas modernas, como el Intel Core i7, utilizan múltiples niveles de caché (L1, L2 y L3 compartida) para optimizar este proceso.

5.5. Procesador

5.5.1. Arquitectura de procesadores

Hasta la década de los 70 la arquitectura de procesadores estaba orientado primordialmente al hardware y a partir de los 80 la tendencia viró hacia una orientación al software. Esta evolución dio lugar a dos filosofías arquitectónicas contrapuestas: *CISC* y *RISC*.

5.5.2. CISC (Complex Instruction Set Computer)

La arquitectura *CISC* se caracteriza por tener un set de instrucciones amplio y complejo, donde las instrucciones suelen requerir **más de un ciclo de reloj** para ejecutarse. Por esto, requiere una microarquitectura de software/hardware muy compleja para decodificar sus múltiples formatos de instrucción. Utiliza pocos registros especializados, maneja muchos tipos de datos y formatos de instrucción variables, y está fuertemente orientada al hardware. Esto permite que los compiladores sean relativamente simples y el código resultante sea pequeño. Ejemplos clásicos incluyen el VAX, la familia Intel x86 e Intel-64, así como los mainframes de IBM.

5.5.3. RISC (Reduced Instruction Set Computer)

Por el contrario, la arquitectura *RISC* apuesta por la simplicidad con un set de instrucciones pequeño y formatos fijos, garantizando una **microarquitectura de hardware** simple. Aquí, las instrucciones son simples, ejecutándose en un solo ciclo de reloj, y el acceso a memoria se limita estrictamente a las operaciones LOAD y STORE. A diferencia de CISC, cuenta con muchos registros de uso general y está orientada al software, lo que exige **compiladores más complejos** y genera un tamaño de código más largo. Esta arquitectura se observa en procesadores como SPARC, MIPS, ARM e Intel Itanium.

5.5.4. Rendimiento y Taxonomía

Para medir la capacidad de estos sistemas, se utiliza la *ecuación de performance*, donde la tasa de MIPS (Millones de Instrucciones Por Segundo) se define como el producto: frecuencia del reloj (en MHz) · instrucciones por ciclo (IPC)

En cuanto a la organización de los sistemas, la **Taxonomía de Flynn** clasifica los procesadores en cuatro categorías:

- **SISD:** Una sola instrucción y un solo dato (típico de uniprosesadores).
- **SIMD:** Una sola instrucción que opera sobre múltiples datos.
- **MISD:** Múltiples instrucciones sobre un solo dato (sin uso comercial).
- **MIMD:** Múltiples instrucciones sobre múltiples datos.

Es importante notar que el incremento de la velocidad del reloj en los procesadores ha encontrado una barrera física crítica: la limitación por el **calor** generado.

5.5.5. Técnicas de Paralelismo

Para superar las limitaciones físicas y mejorar el rendimiento, se han desarrollado diversas técnicas de paralelismo divididas en dos niveles:

Paralelismo a Nivel de Instrucción

Este nivel busca solapar la ejecución de instrucciones para reducir el tiempo total de procesamiento.

- **Pipelining (Stages):** Divide la ejecución en etapas (*fetch*, *decode*, *operand fetch*, *execution* y *write back*), con el objetivo de reducir el tiempo total de una secuencia de instrucciones, permitiendo ejecutar una instrucción por ciclo de reloj, como en el Intel 486. El control de las dependencias entre las instrucciones puede resolverlo tanto el compilador como el hardware.

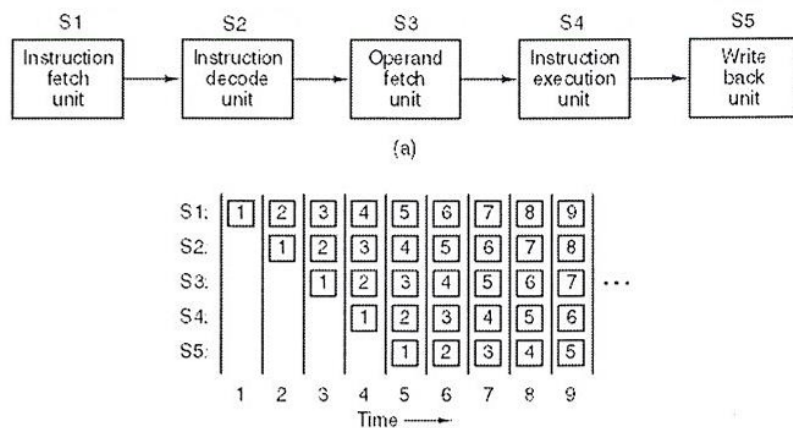


Figura 24: Técnica de Pipelining

- **Dual Pipelining:** Duplica las unidades de ejecución para procesar dos instrucciones por ciclo, técnica implementada en el Intel Pentium.

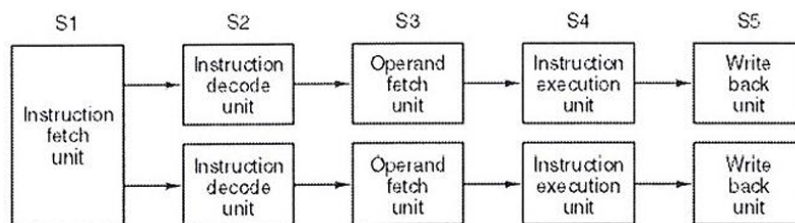


Figura 25: Técnica de Dual Pipelining

- **Superscalar:** Utiliza múltiples unidades funcionales para ejecutar más de una instrucción por ciclo (N-way o N-issue), siendo N un valor típicamente entre 3 y 6, característico de la arquitectura Intel Core.

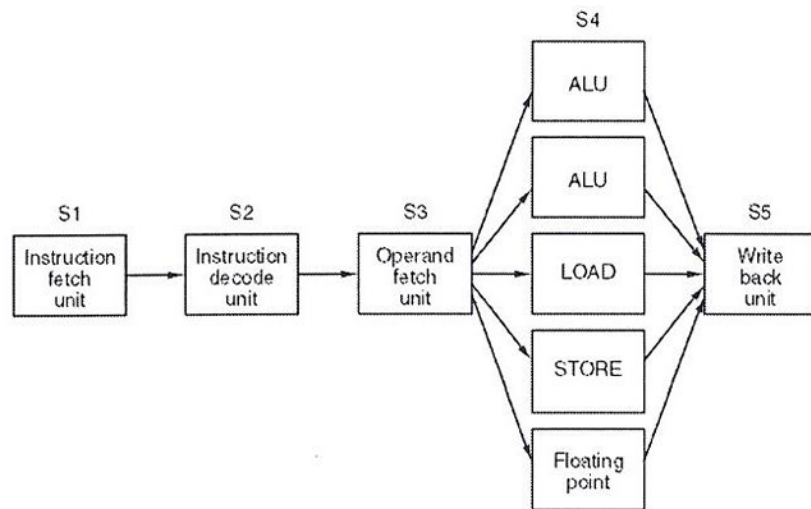


Figura 26: Técnica de Superscalar

- **Hardware Multithreading:** consiste en intercambiar la ejecución entre diferentes threads (hilos de ejecución) cuando uno de ellos se encuentra detenido o frenado por alguna causa externa (como una espera de datos). Los conceptos fundamentales son
 - **Thread:** Se define como un “proceso liviano”. Contiene su propio Contador de Programa (PC), un conjunto de registros y una pila (*stack*), pero comparte el mismo espacio de direccionamiento con otros hilos.
 - **Proceso:** Puede tener uno o más threads, y contiene un address space y un estado que es gestionado por el sistema operativo.
 - **Eficiencia:** El cambio de contexto entre threads es mucho más rápido que entre procesos, ya que se realiza en un mismo ciclo de reloj sin requerir la intervención directa del sistema operativo.

Existen dos enfoques principales:

1. **Fine-grained:** El procesador alterna entre los hilos **después de ejecutar cada instrucción**. Un ejemplo de esto son los procesadores Intel IA-32.
2. **Coarse-grained:** El intercambio solo ocurre ante un **evento significativo**, como un fallo de página (*page fault*) o un fallo de caché (*cache miss*). En lugar de esperar el acceso lento a la memoria principal, el CPU pasa a ejecutar otro hilo. El Intel Itanium 2 utiliza este método.

Paralelismo a Nivel de Procesador

- **Procesadores paralelos de datos:** Utilizan una sola unidad de control para múltiples procesadores. Aquí destaca el método **SIMD**, donde se ejecutan los mismos pasos sobre distintos datos (común en *GPUs* como Nvidia Fermi) y los **procesadores vectoriales**, que operan mediante *pipelining* sobre registros vectoriales (conjunto de registros convencionales que se cargan desde memoria en una sola instrucción).
- **Multiprocesadores:** Son sistemas fuertemente acoplados (**MIMD**) que comparten memoria. Distintas implementaciones: pueden ser de acceso uniforme (**UMA/SMP**, con bus único y memoria centralizada, como el Intel Core i7) o de acceso no uniforme (**NUMA**, con memoria local y compartida, como el SGI Origin 2000).

- **Multicomputadores:** Son sistemas ligeramente acoplados (**Clusters**) donde cada computador tiene su propia memoria local distribuida y se comunican mediante intercambio de mensajes. No existe memoria compartida y suelen utilizar topologías de grillas, árboles o anillos, siendo el IBM Blue Gene/P un ejemplo emblemático de esta arquitectura escalable.

6. Unidad 6: Almacenamiento Secundario

6.1. Cintas Magnéticas

6.1.1. Información General

Las cintas magnéticas constituyen uno de los medios de almacenamiento secundario más tradicionales. **Físicamente**, consisten en un soporte de poliéster flexible recubierto de material magnetizable, presentándose tanto en carretes abiertos como en paquetes cerrados o cartuchos. Sus dimensiones estándar de ancho varían entre los 0.38 cm (0.25 pulgadas) y los 1.27 cm (0.5 pulgadas)

Una de sus características fundamentales es el *acceso secuencial* a la información. Esto implica que, para pasar de un registro inicial a uno distante, es necesario “leer” todos los registros intermedios; de igual forma, para acceder a un dato anterior, la unidad debe rebobinar la cinta y reiniciar la búsqueda.

6.1.2. Técnicas de Grabación

A lo largo de su evolución, se han implementado diversas técnicas para el registro de datos:

1. **Grabación en Paralelo:** Fue la técnica original, donde una cabeza de grabación estacionaria registraba pistas de forma paralela a lo largo de la cinta. Inicialmente se utilizaban 9 pistas (8 para datos y 1 de paridad para detección de errores), evolucionando luego a 18 para palabras y 36 para doble palabra. Modelos históricos de IBM, como el 726 (1953) o el 3480 (1985), muestran esta evolución en densidad lineal y capacidad, pasando de carretes de vacío a cartuchos.
2. **Grabación en Serie (Serpentina):** Es un sistema más moderno que también utiliza una cabeza estacionaria. Los datos se escriben a lo largo de una pista hasta el final de la cinta y luego se continúa en sentido contrario por una pista adyacente. Este método permite grabar de 2 a 8 pistas en simultáneo (multitrack).
3. **Estándar LTO (Linear Tape Open):** Creado en 1997 por HP, IBM y Seagate, este estándar de grabación en serie ha definido el camino del almacenamiento masivo. Desde el LTO-1 (100 GB nativos) hasta las proyecciones para LTO-12 (192 TB nativos), este formato ofrece capacidades de compresión, cifrado y tecnología WORM. Actualmente, se utiliza a través de cartuchos de datos, unidades externas y bibliotecas de cintas (Tape Libraries) de gran escala.
4. **Grabación Helicoidal:** Similar al funcionamiento de las videocasetas (VCR), utiliza una cabeza rotatoria que gira rápidamente mientras la cinta se mueve lento. Esto permite que las pistas estén más cerca entre sí y evita el desgaste por altas velocidades de cinta. Sus formatos conocidos incluyen DAT/DDS, AIT y Exabyte Mammoth.

6.1.3. Modos de Operación y Usos

Existen **dos modos principales de operación**. El modo *start-stop* por bloque es un uso antiguo donde se grababan archivos en bloques físicos separados por espacios de sincronización (IRG – Inter Record Gap), permitiendo actualizar bloques individuales si no cambiaban de tamaño. Por otro lado, el modo *streaming* se utiliza para backup, escribiendo archivos completos como un flujo de datos contiguo sin necesidad de localizar bloques particulares.

En conclusión, aunque fue el primer medio de almacenamiento secundario y es el **nivel más lento** en la jerarquía de memoria, la cinta magnética sigue vigente para el backup y archivo de información a largo plazo (más de 30 años) debido a su **bajo costo por byte y gran capacidad**. Físicamente, su gestión se apoya en marcas críticas como el BOT (comienzo de cinta) y el EOT (fin de cinta).

6.2. Discos Magnéticos

Estos dispositivos se basan en un medio físico compuesto por un **plato circular construido sobre un substrato no magnético**, usualmente de aluminio o vidrio, el cual se encuentra **recubierto por un material magnetizable**.

6.2.1. Mecanismos de Lectura y Escritura

El funcionamiento de estos discos ha evolucionado significativamente. En los **modelos antiguos** y disquetes, se utilizaba una cabeza de lectura/escritura única consistente en una bobina conductora estática, bajo la cual el disco gira constantemente. El proceso de escritura se basa en la circulación de electricidad por la bobina para producir un campo magnético que graba patrones en la superficie, mientras que la lectura detecta la corriente eléctrica generada cuando la superficie magnetizada pasa bajo la cabeza.

Los **sistemas modernos** emplean una cabeza de lectura diferenciada de la de escritura. La *cabeza de lectura* utiliza un sensor magneto-resistivo (MR), cuya resistencia eléctrica varía según la dirección de magnetización del medio. Al pasar una corriente por el sensor, los cambios de resistencia se detectan como señales de voltaje, lo que permite mayores densidades de grabación y velocidades de operación.

6.2.2. Organización y Estructura del Disco

La *superficie del disco* se organiza en pistas concéntricas, cuyo ancho coincide con el de la cabeza lectora/grabadora. Para minimizar interferencias y errores de desalineamiento, se implementan espacios denominados **Intertrack** gaps entre pistas. Asimismo, cada pista se subdivide en sectores de tamaño fijo, usualmente de 512 bytes, separados por un Intersector gap para evitar errores de sincronización.

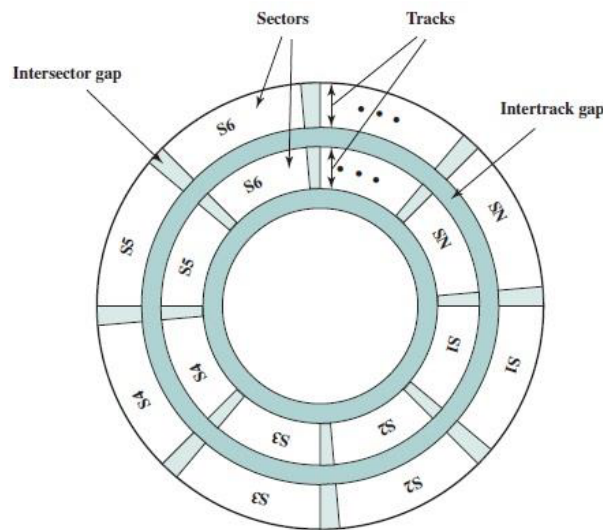


Figura 27: Organización del Disco

En cuanto al *método de grabación*, existen dos enfoques principales:

1. **CAV (Constant Angular Velocity):** El disco gira a velocidad constante. Aunque facilita el referenciamiento por pista/sector, los bits exteriores están más espaciados para compensar su mayor velocidad lineal, lo que resulta en una menor densidad de bits. Sus *ventajas* son su **mecánica y electrónica simples**, tiempos de acceso **rápidos** y transferencia **predecible**. Sus *desventajas* son el **desperdicio de espacio** y la **menor capacidad** total.
2. **Grabación Multizona:** La superficie se divide en zonas concéntricas (típicamente 16). Las zonas exteriores contienen más sectores, aprovechando mejor la capacidad de almacenamiento, aunque requiere una circuitería más compleja para manejar distintos tiempos de lectura/escritura. Sus *ventajas* son la **maximización de la capacidad** de almacenamiento y su **mayor velocidad en zonas externas**. Sus *desventajas* son su **complejo controlador** y un **rendimiento inconsistente**.

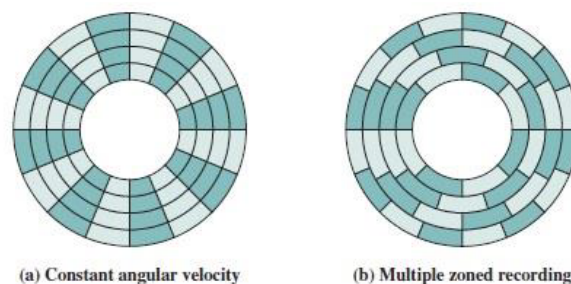


Figura 28: Métodos de Grabación, gráficamente

Físicamente, un sector se compone de diversos campos, incluyendo bytes de sincronización, identificación de pista, cabeza y sector, seguidos por los 512 bytes de datos y un código de redundancia cíclica (CRC).

6.2.3. Características Físicas y de Rendimiento

La *estructura física* de un disco magnético define su capacidad y durabilidad. Un factor determinante es el movimiento de la cabeza: mientras que los sistemas de “cabeza fija” (una por pista) quedaron en el pasado por su altísimo costo (como el IBM 2305), los **sistemas actuales** de cabeza móvil utilizan un solo cabezal por superficie, montado en un brazo mecánico que se desplaza radialmente.

En cuanto a su *construcción*, existen diversas variantes:

- **Portabilidad:** Los discos rígidos tradicionales son no removibles y están montados permanentemente en un disk drive, a diferencia de los antiguos floppy disks, que podían retirarse de la unidad (son removibles).
- **Configuración de Platos y Lados:** Un disco puede tener una o dos caras con recubrimiento magnético. Cuando el sistema utiliza múltiples platos, se introduce el concepto crítico de *cilindro*: el conjunto de todas las **pistas que se encuentran a la misma distancia del centro** en todos los platos disponibles.
- **Mecanismo de la Cabeza:** El cabezal puede operar por contacto (desgastando la superficie, como en los disquetes), a una posición fija sobre el disco, o mediante un espacio aerodinámico (flotante). Este último es el más avanzado: la propia **rotación del disco genera una presión de aire** que hace “volar” la cabeza a una distancia mínima de la superficie sin tocarla.

6.2.4. Parámetros de Performance

El desempeño no depende solo del hardware, sino también del sistema operativo y el controlador. Para medirlo, el *tiempo total de una operación* se divide en etapas críticas:

- **Tiempo de Seek (Búsqueda):** Es el tiempo que tarda el brazo mecánico en mover la cabeza lectora exactamente sobre la pista deseada. Es, generalmente, el componente más lento.
- **Latencia Rotacional (Demora):** Una vez que la cabeza está en la pista, debe esperar a que el disco gire hasta que el sector específico pase por debajo de ella.
- **Tiempo de Acceso:** Es la suma del seek promedio y la latencia promedio. Es el indicador principal de la agilidad del disco para posicionarse.
- **Tiempo de Transferencia (T_r):** Una vez posicionado, es el tiempo que toma leer los datos. Se rige por la fórmula $T_r = \frac{b}{rN}$, donde influyen los bytes a transferir (b), la velocidad de rotation (r) y la densidad de datos por pista (N).

El tiempo total de lectura/escritura se consolida mediante la fórmula matemática:

$$T = T_{seek} + \frac{1}{2r} + \frac{b}{rN}$$

Donde el término $\frac{1}{2r}$ representa la *latencia rotacional promedio* (medio giro del disco).

6.2.5. RAID (Redundant Array of Independent Disks)

Cuando un solo disco no es suficiente, se utilizan arreglos RAID, un conjunto de discos que operan de forma independiente y en paralelo, y que el sistema operativo ve como una única unidad lógica. Como los discos son independientes entre sí, un pedido de entrada y salida puede resolverse en paralelo si los datos involucrados residen en discos separados, lo que le da a RAID una ventaja de rendimiento además de la de confiabilidad. Los datos se distribuyen mediante *striping* (división en franjas) para ganar velocidad o redundancia, según el nivel utilizado. RAID es un estándar que consiste en 7 niveles, del 0 al 6, y también **pueden implementarse combinaciones entre niveles**, por ejemplo RAID 0+1 o RAID 5+0, para combinar las ventajas de ambos esquemas.

RAID 0 (stripping): no incluye redundancia. Requiere N discos y distribuye los datos en *strips*, que pueden ser sectores, bloques u otra unidad. Sus ventajas son

- La simplicidad
- La performance

Su desventaja es clara: frente a fallos existe riesgo total, ya que no hay posibilidad de recuperación.

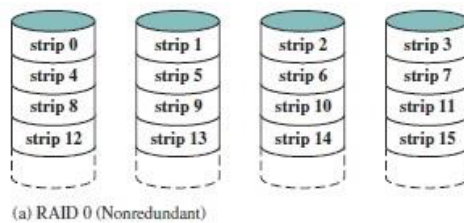


Figura 29: Raid 0

RAID 1 (espejado): introduce redundancia mediante la copia exacta de los datos. Requiere 2N discos. Entre sus ventajas se menciona que:

- Un pedido de lectura puede resolverse desde **cualquiera de los dos** discos
- La **escritura se realiza en forma independiente** en cada disco sin penalización significativa.
- La **recuperación ante fallas es simple** y que ofrece alta disponibilidad de datos.

Su principal desventaja es el costo.

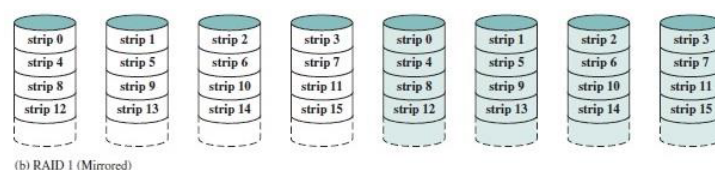


Figura 30: Raid 1

RAID 2 utiliza redundancia por código autocorrector. Trabaja con *strips* pequeños, de un byte o una palabra. Requiere N más m discos, grabando bits de paridad en discos separados. Todos los dos discos se leen y escriben en paralelo, en forma sincronizada. Se aclara que no existe uso comercial. Su ventaja es la disponibilidad de datos, mientras que la desventaja son los costos asociados al método de redundancia.

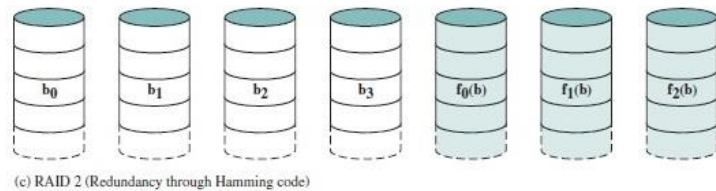


Figura 31: Raid 2

RAID 3 emplea paridad por intercalamiento de bits. Usa un único disco de paridad y requiere $N+1$ discos. La paridad se calcula bit a bit a través del conjunto de bits de la misma posición en todos los discos. Al igual que en RAID 2, los discos se leen y escriben en paralelo y de manera sincronizada. Entre sus *ventajas* figuran:

- Sencillez del cálculo de paridad.
- Ausencia de un impacto significativo en performance ante fallas.

Considerando un caso de cinco discos en total. Sea X_5 el disco de paridad y $X_i, i \in \mathbb{I}_4$ los discos que contienen los datos. Luego,

$$X_5 = X_1 \otimes X_2 \otimes X_3 \otimes X_4$$

Siendo $\otimes$ la operación XOR.

Suponiendo que necesitamos recuperar los datos de X_1 , hacemos:

$$\begin{aligned} X_5 &= X_1 \otimes X_2 \otimes X_3 \otimes X_4 \\ X_1 \otimes X_5 &= X_1 \otimes X_2 \otimes X_3 \otimes X_4 \otimes X_1 \\ X_1 &= X_2 \otimes X_3 \otimes X_4 \otimes X_5 \end{aligned}$$

Como desventaja aparece la complejidad del controlador.

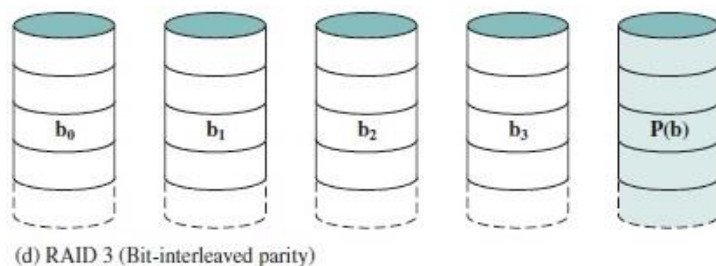


Figura 32: Raid 3

RAID 4 se basa en paridad por intercalamiento de bloques. En este nivel se accede en forma independiente a cada disco, se requieren $N+1$ discos y puede darse servicio a

pedidos de entrada y salida en paralelo. Utiliza *strips* grandes. La paridad se calcula de manera similar a RAID 3, pero se guarda un *strip* de paridad. También se aclara que no tiene uso comercial. Su principal ventaja es ofrecer altas tasas de lectura. Entre sus desventajas están:

- Necesidad de realizar dos lecturas y dos escrituras ante una actualización de datos.
- Generación de un cuello de botella en el disco de paridad.

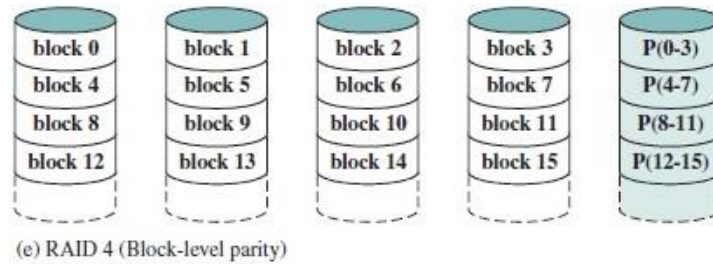


Figura 33: Raid 4

RAID 5 mejora este punto mediante paridad por intercalamiento distribuido de bloques. También accede en forma independiente a cada disco y requiere N+1 discos, pero en este caso los strips de paridad se distribuyen entre todos los discos. Su gran *ventaja* es que **resuelve el cuello de botella** característico de RAID 4. Como *desventaja*, nuevamente, aparece la complejidad del controlador.

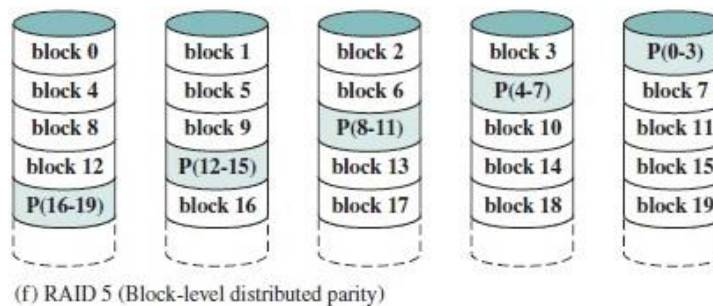


Figura 34: Raid 5

RAID 6 agrega doble paridad por intercalamiento distribuido de bloques. Se accede de manera independiente a cada disco, pero se requieren N+2 discos. Para ello se usan dos algoritmos de control de paridad. Las *ventajas* principales son que provee una **disponibilidad de datos extremadamente alta** y es **más tolerante** ante fallos. Sus *desventajas* son la **complejidad del controlador** y el **mayor costo** derivado de la doble paridad.

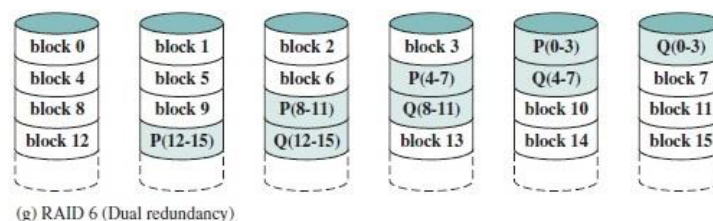


Figura 35: Raid 6

6.3. Medios Ópticos

6.3.1. CD (Compact Disc)

El punto de partida es el CD, cuyo estándar internacional fue definido en 1980 mediante un acuerdo entre Philips y Sony, documentado en el llamado *Red Book*. Físicamente, el disco mide 120 mm de ancho y 1,2 mm de espesor, y su funcionamiento se basa en una velocidad lineal constante —CLV, por sus siglas en inglés— sobre una pista única dispuesta en forma de espiral. La unidad mínima de almacenamiento es el sector, que ocupa 2352 bytes.

El proceso de fabricación del disco maestro (*master*) comienza con un láser infrarrojo de alto poder que marca huecos sobre un disco de vidrio. A partir de ese molde se inyecta policarbonato, que replica fielmente los patrones de los huecos. Sobre ese sustrato se deposita una capa reflectiva de aluminio y, finalmente, una capa protectora de laca junto con una etiqueta impresa.

En cuanto a la representación de datos, se distinguen dos elementos físicos: el *pit*, que es la marca física o hueco, y el *land*, que es la superficie plana sin marcas. La codificación binaria se realiza por transiciones: un bit en **1** corresponde a una transición entre *pit* y *land*, o viceversa; mientras que un bit en **0** representa la ausencia de transición durante un tiempo determinado, lo que hace indispensable el uso de CLV para garantizar una medición temporal uniforme.

6.3.2. CD-ROM (Compact Disc-Read Only Memory)

En 1984 se estableció el estándar *Yellow Book*, que definió el CD-ROM (*Compact Disc-Read Only Memory*). Este formato incorporó el esquema de codificación **EFM** (es una forma de codificar datos binarios, utilizada en medios ópticos. Se usa para transformar los datos binarios originales en un patrón de bits que sea físicamente grabable y leíble por un láser. Es necesaria porque elige, de todos los códigos de 14 bits posibles, solo los que garantizan que entre dos unos consecutivos haya un mínimo y un máximo de ceros — ni transiciones pegadas ni demasiado espaciadas.) —*Eight to Fourteen Modulation*— y definió dos modos de operación. El **Modo 1**, orientado a datos, asigna 2048 bytes útiles por sector. El **Modo 2**, pensado para audio, amplía ese espacio a 2336 bytes, lo que permite almacenar hasta 74 minutos de contenido sonoro. El sistema de archivos empleado en el Modo 1 es el **ISO 9660**.

La estructura interna del formato se organiza jerárquicamente: los *símbolos*, de 14 bits cada uno, se agrupan en bloques de 42 para conformar un *frame* de 588 bits que contiene 24 bytes de datos. A su vez, 98 frames componen un *sector de Modo 1*, cuyo tamaño total es de 2352 bytes, distribuidos en 16 bytes de preámbulo, 2048 bytes de datos y 288 bytes de corrección de errores (ECC).

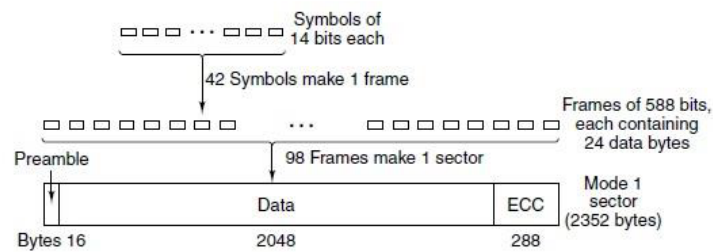


Figura 36: CD-ROM

6.3.3. CD-R (Compact Disc-Recordable)

El *CD-R* permitía escribir una única vez. En lugar de los huecos físicos del CD convencional, el CD-R **utiliza una capa de tinta** que en su estado inicial es transparente y permite el paso del láser. Al escribir, el **láser quema esa capa** creando puntos oscuros que no reflejan la luz, simulando así el comportamiento de un *pit*. Este formato dio origen también a la variante CD-XA.

6.3.4. CD-RW (Compact Disc-Rewritable)

El *CD-RW* (*Compact Disc-Rewritable*) introdujo la posibilidad de grabar y borrar múltiples veces, gracias a un material con dos estados físicos de reflectividad distinta: el estado **cristalino**, que refleja la luz, y el estado **amorfo**, que no lo hace.

El proceso diferencia tres niveles de potencia láser. Para **escribir**, se aplica alta potencia y se lleva el material del estado cristalino al amorfo, representando un *pit*. Para **borrar**, se usa potencia media y se revierte el proceso, volviendo al estado cristalino. Para **leer**, basta con baja potencia, sin alterar el estado del material.

6.3.5. DVD (Digital Versatile Disk)

El *DVD* comparte el **diseño general del CD**, pero fue concebido específicamente para la industria de **distribución de video**. Sus mejoras técnicas son significativas: los *pits* son más pequeños ($0,4\ \mu\text{m}$ frente a $0,8\ \mu\text{m}$ del CD), la espiral está más comprimida ($0,74\ \mu\text{m}$ entre pistas frente a $1,6\ \mu\text{m}$), y se emplea un láser de color rojo con longitud de onda de $0,65\ \mu\text{m}$ —frente a los $0,78\ \mu\text{m}$ del CD—. Todo ello eleva la capacidad base a **4,7 GB**.

El estándar define cuatro configuraciones posibles: un solo lado con una capa (4,7 GB), un solo lado con dos capas (8,5 GB), dos lados con una capa (9,4 GB) y doble lado con doble capa, que alcanza los **17 GB**. Esta última configuración apila dos sustratos de policarbonato de $0,6\ \text{mm}$ cada uno, unidos por una capa adhesiva, con capas reflectivas de aluminio y semirreflectivas distribuidas entre ellas.

En cuanto a sus variantes regrabables, el **DVD-R** emplea tecnología análoga al CD-R para grabaciones únicas, mientras que el **DVD-RW** adopta la lógica del CD-RW para permitir múltiples ciclos de escritura y borrado.

6.3.6. Medios de Alta Definición: HD DVD y Blu-ray

Para la distribución de video de alta definición se desarrollaron dos estándares en competencia: el **HD-DVD** (*High Definition Digital Versatile Disk*) y el **Blu-ray Disc (BD)**. Finalmente, fue el Blu-ray la tecnología que se impuso en el mercado.

El Blu-ray se distingue por el uso de un láser azul de $0,405\ \mu\text{m}$ de longitud de onda, lo que permite densidades de grabación sensiblemente mayores. Su capacidad es de **25 GB** en la versión de una sola capa y de **50 GB** en la de doble capa. Al igual que sus predecesores, cuenta con variantes de escritura única (**BD-R**) y de regrabado múltiple (**BD-RE**).

6.3.7. Tecnología “Archival Disc” de Sony y Panasonic

Orientada al almacenamiento de largo plazo, la tecnología **Archival Disc** fue desarrollada conjuntamente por Sony y Panasonic, y se lanzó en tres generaciones sucesivas.

La primera generación (2015) ofrece 300 GB de capacidad en un disco de doble lado (*double-sided*) con tres capas por cara. Introduce la grabación tanto en *land* como en *groove* —es decir, en la zona plana y en el surco de la espiral—, con una longitud de onda de 405 nm y una distancia entre pistas de $0,225\ \mu\text{m}$. Su estructura interna incorpora películas dieléctricas que separan las capas de grabación.

La segunda generación eleva la capacidad a 500 GB, manteniendo las características de la primera y añadiendo alta densidad lineal junto con tecnología de cancelación de interferencias entre símbolos.

La tercera generación alcanza **1 TB** por disco, incorporando además tecnología de grabación multinivel, que permite almacenar más de un bit por marca.

6.3.8. Sony “Optical Disc Archive”

Sobre la base del *Archival Disc*, Sony desarrolló el sistema **Optical Disc Archive**: una solución de almacenamiento masivo que organiza los discos en *cartridges* con capacidad de 3,3 TB cada uno. El sistema es escalable: la unidad maestra (*ODS-L30M*) admite hasta 2 unidades lectoras y 30 *cartridges*, y puede expandirse mediante unidades de extensión (*ODS-L60E* y *ODS-L100E*) hasta alcanzar configuraciones de 535 *cartridges* y una capacidad total de **1,7655 PB** (*petabytes*).

6.4. SSD

Un **SSD** (*Solid State Drive*) se define como un dispositivo de almacenamiento secundario construido íntegramente con componentes electrónicos de estado sólido, es decir, semiconductores. A diferencia de los discos magnéticos, no posee partes mecánicas móviles, lo que le confiere una serie de características distintivas tanto en rendimiento como en durabilidad.

Su historia se remonta a dos líneas tecnológicas bien diferenciadas. La primera generación de SSDs estaba basada en memoria **RAM**, lo que los hacía volátiles: necesitaban una

fuelle de energía auxiliar para retener los datos. Un ejemplo temprano fue el desarrollado por Texas Memory en 1978, con una capacidad de apenas 16 KB. La segunda línea, y la que finalmente prevaleció, es la basada en memoria **flash no volátil**, cuyo hito comercial fue el lanzamiento de M-Systems en 1995. Sobre esa base se desarrolló la tecnología que hoy conocemos: la memoria **NAND Flash**, que constituye el núcleo de todos los SSDs modernos.

6.4.1. Arquitectura interna

Internamente, un SSD está compuesto por cuatro elementos principales. El **controlador** es el componente central que gestiona todas las operaciones de lectura, escritura y borrado, además de administrar funciones avanzadas como el nivelado de desgaste. La **caché** es una memoria de acceso rápido que actúa como buffer para optimizar las transferencias de datos. Los **chips de memoria NAND Flash** son los encargados del almacenamiento persistente propiamente dicho, y constituyen la mayor parte del espacio físico de la placa. Por último, el **condensador** cumple una función de respaldo energético que permite completar operaciones de escritura en curso en caso de una pérdida repentina de alimentación, protegiendo así la integridad de los datos.

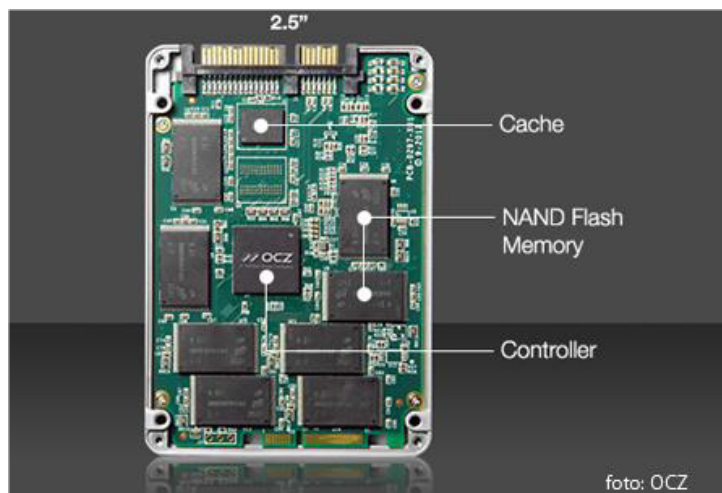


Figura 37: SSD

6.4.2. Comparación con los discos magnéticos

Al confrontar los **SSDs** con los discos magnéticos tradicionales (HDD), la balanza de ventajas se inclina claramente hacia los primeros en la mayoría de los aspectos técnicos y operativos.

Entre las **ventajas de los SSDs** se destacan:

- Arranque significativamente más **rápido** del sistema.
- Gran velocidad tanto de lectura como de escritura.
- **Baja latencia** en ambas operaciones.
- Consumen **menos energía** y generan menos calor.

- Al carecer de partes mecánicas, operan sin **ningún tipo de ruido**.
- En términos de confiabilidad, presentan un mejor *MTBF* (*Mean Time Between Failures*, o tiempo medio entre fallas).
- **Mayor seguridad** en la integridad de los datos.
- **Rendimiento determinístico** —es decir, predecible y estable independientemente del patrón de acceso—.
- A nivel físico, son más **livianos y compactos**.
- Ofrecen una **resistencia muy superior** ante golpes, caídas y vibraciones.

No obstante, los **SSDs** también presentan **desventajas** relevantes frente a los discos magnéticos:

- El **precio** por gigabyte sigue siendo más elevado.
- La **recuperación de datos** ante fallos es más limitada.
- La **vida útil** de las celdas de memoria NAND es limitada, dado que cada celda soporta un número limitado de ciclos de escritura y borrado antes de degradarse.

6.4.3. Tecnologías de conexión

Los **SSDs** pueden clasificarse según tres dimensiones tecnológicas: la interfaz externa, el protocolo de comunicación y el factor de forma físico.

En cuanto a la **interfaz externa**, las dos opciones principales son **SATA** y **PCIe**. **SATA** es la tecnología más antigua y compatible, con velocidades que evolucionaron desde los 150 MB/s de SATA I hasta los 600 MB/s de SATA III. **PCIe** es considerablemente más rápido: en su versión Gen 4 con 16 canales (*lanes*), puede transferir datos a 32.000 MB/s.

Respecto al **protocolo de comunicación**, existen dos estándares: **AHCI** y **NVMe**. **AHCI** fue diseñado originalmente para discos duros con tecnología de platinos giratorios, y por eso presenta limitaciones cuando se lo emplea con **SSDs**: soporta una única cola de comandos con un máximo de 32 comandos simultáneos, utiliza muchos ciclos de CPU y tiene una latencia de 6 microsegundos, con un tope de 100K IOPS. **NVMe**, en cambio, fue diseñado específicamente para dispositivos flash: admite hasta 64.000 colas de comandos con 64.000 comandos por cola, utiliza pocos ciclos de CPU, tiene una latencia de apenas 2,8 microsegundos y puede superar el millón de IOPS. A diferencia de AHCI, **NVMe** se comunica directamente con la CPU del sistema, eliminando la intermediación del controlador SATA.

Finalmente, en lo que respecta al **factor de forma** (*form factor*), los **SSDs** se presentan en distintos formatos físicos según la interfaz que utilizan. En la familia SATA se encuentran el **formato 2.5"** —el más común y compatible con bahías de laptops y desktops—, el **mSATA** —diseñado para sistemas de menor tamaño— y el **M.2 con soporte AHCI**. En la familia PCIe se dispone del formato **HHHL** (*Half Height, Half Length*, también llamado AIC o *Add-In Card*), del **M.2 con soporte NVMe** y del **U.2**, este último disponible exclusivamente en la variante NVMe.

6.4.4. Tabla Comparativa con Ventajas y Desventajas de Cada Medio

Medio	Ventajas	Desventajas
Cintas magnéticas	• Primer medio de almacenamiento secundario; tecnología madura y probada • Muy bajo costo por byte almacenado • Alta capacidad por cartucho (hasta 12 TB nativos en LTO-8; 30 TB comprimidos) • Larga durabilidad del medio físico (30 años o más) • Escalabilidad masiva mediante librerías de cintas (hasta 1,6 EB) • Ideal para backup y archivado a largo plazo • Soporte WORM para integridad de datos	• Acceso estrictamente secuencial: lento para datos aleatorios • Requiere rebobinar para acceder a registros anteriores • Medio más lento en la jerarquía de memoria • En modo streaming no se pueden localizar ni modificar bloques individuales • Requiere infraestructura específica (drives y librerías)
Discos magnéticos (HDD)	• Acceso aleatorio: permite referenciar cualquier bloque por pista/sector • Gran capacidad de almacenamiento (hasta 12 TB+ en modelos actuales) • Bajo costo por GB en comparación con SSD • Posibilidad de recuperación de datos ante fallos físicos • Compatible con RAID para redundancia y mayor rendimiento • Tecnología ampliamente disponible y soportada	• Latencia mecánica: tiempo de seek + latencia rotacional • Partes móviles que generan ruido, calor y puntos de falla • Mayor consumo de energía que SSD • Sensible a golpes, caídas y vibraciones (resistencia hasta 350 g/2 ms) • Rendimiento variable según la posición del dato en el disco (CAV) • Mayor tamaño y peso frente a medios más modernos
Medios ópticos (CD/DVD/Blu-ray)	• Alta portabilidad y facilidad de transporte • Bajo costo por unidad para distribución masiva de contenido • Buena durabilidad del medio si se almacena correctamente • Estándares abiertos e interoperables (Red Book, Yellow Book, etc.) • Variantes de solo lectura (ROM), grabación única (R) y regrabables (RW/RE) • Archival Disc permite hasta 1 TB por disco con gran durabilidad	• Capacidad limitada frente a HDD y SSD (máx. 50 GB en Blu-ray estándar) • Velocidades de lectura/escritura bajas • Susceptibles a rayaduras, polvo y condiciones ambientales adversas • Los formatos grabables (CD-R, DVD-R) solo permiten una escritura • Lectores/grabadores cada vez menos presentes en dispositivos modernos • Degradación del medio si no se almacena en condiciones óptimas
SSD (Solid State Drive)	• Arranque y acceso a datos muy rápidos; alta velocidad de lectura y escritura • Baja latencia de acceso (NVMe: 2,8 μs; más de 1 millón de IOPS) • Sin partes móviles: silencioso, menor consumo y menor generación de calor • Alta resistencia a golpes y vibraciones (hasta 1500 g/0,5 ms) • Mejor MTBF y rendimiento determinístico independiente del patrón de acceso • Formato compacto y liviano; múltiples factores de forma disponibles	• Mayor precio por GB respecto a HDD y cintas • Vida útil limitada por ciclos de escritura/borrado de celdas NAND • Capacidades máximas inferiores a los discos magnéticos de alta gama • Menor recuperabilidad ante fallos: modos de falla menos predecibles • Requiere NVMe (no AHCI) para aprovechar su máximo rendimiento